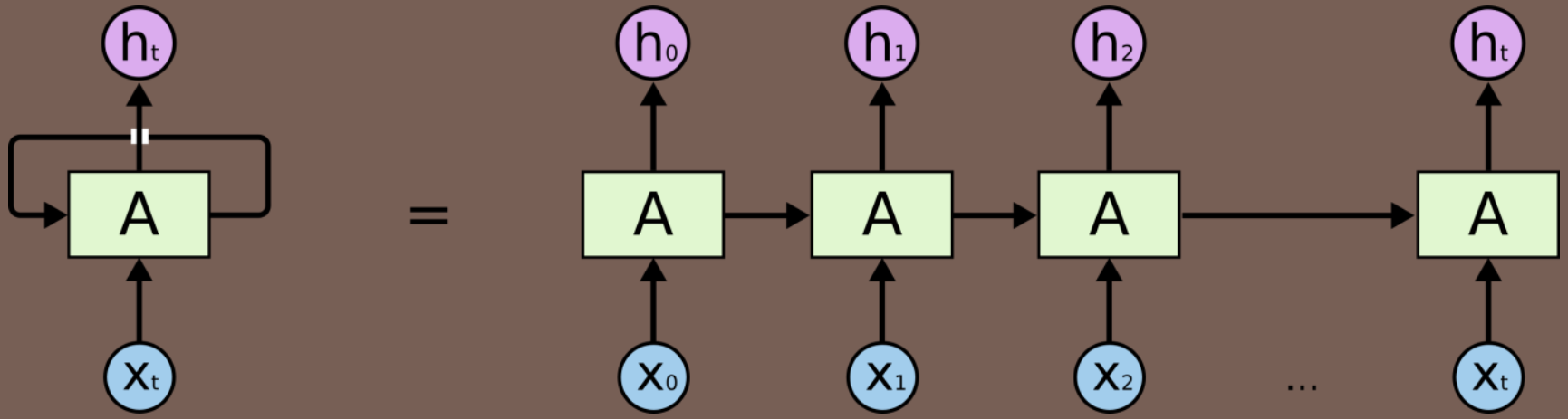
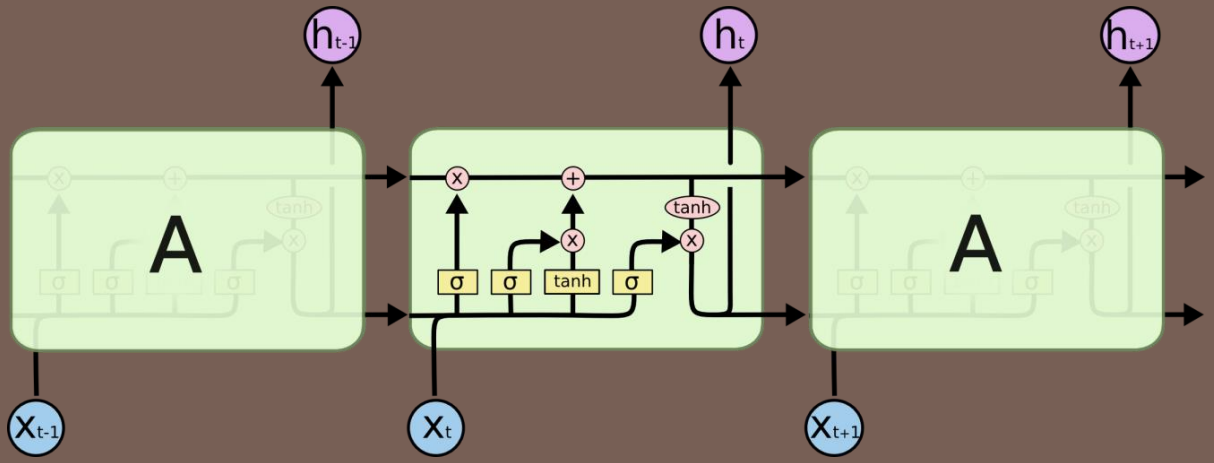
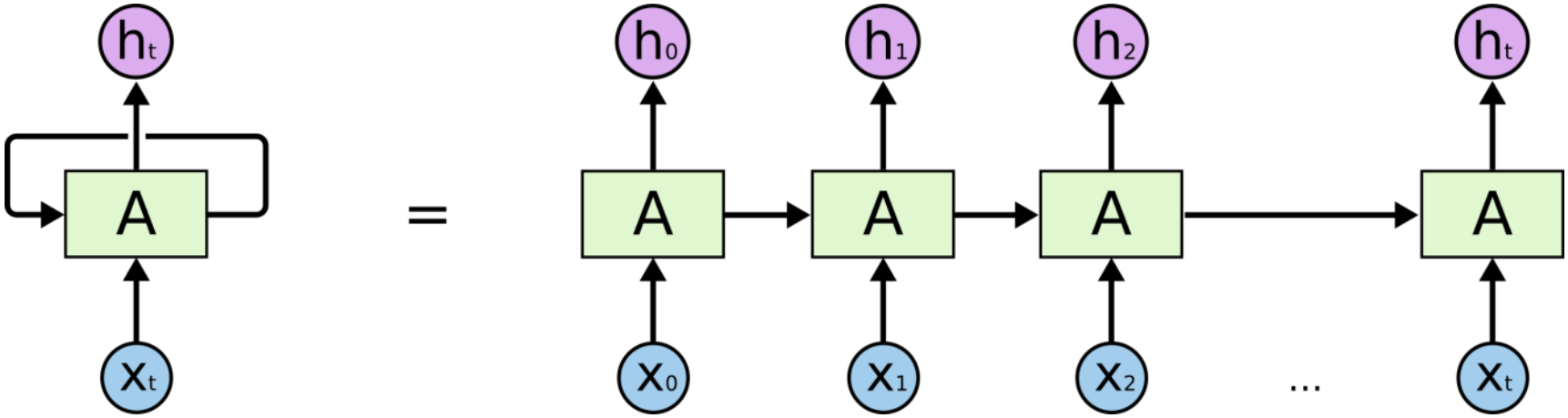

ht
A
Xt
=
h0
h1
h2
ht
A
A
A
A
X0
X1
X2
...
Xt


ht-1
ht
ht+1
A
A
tanh
σ
σ
tanh
σ
Xt-1
Xt
Xt+1

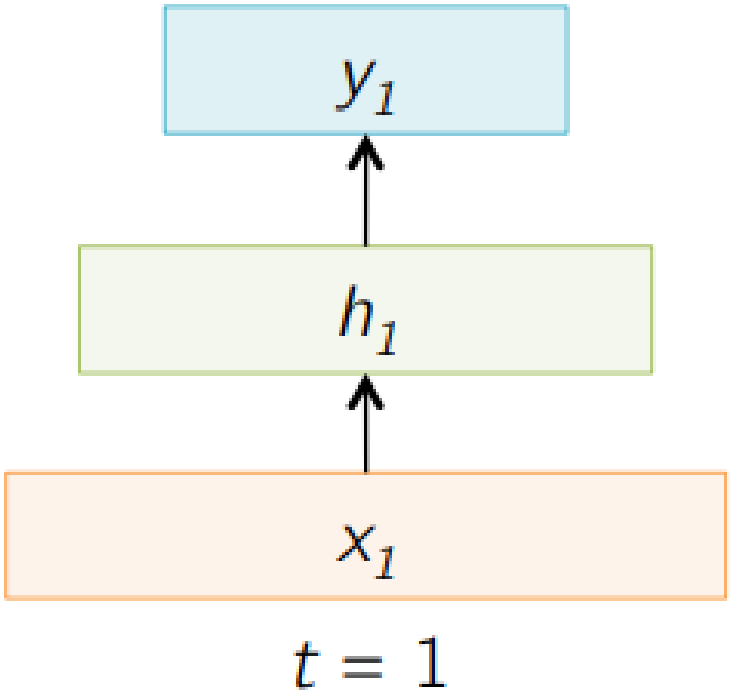
RNN

RNNs

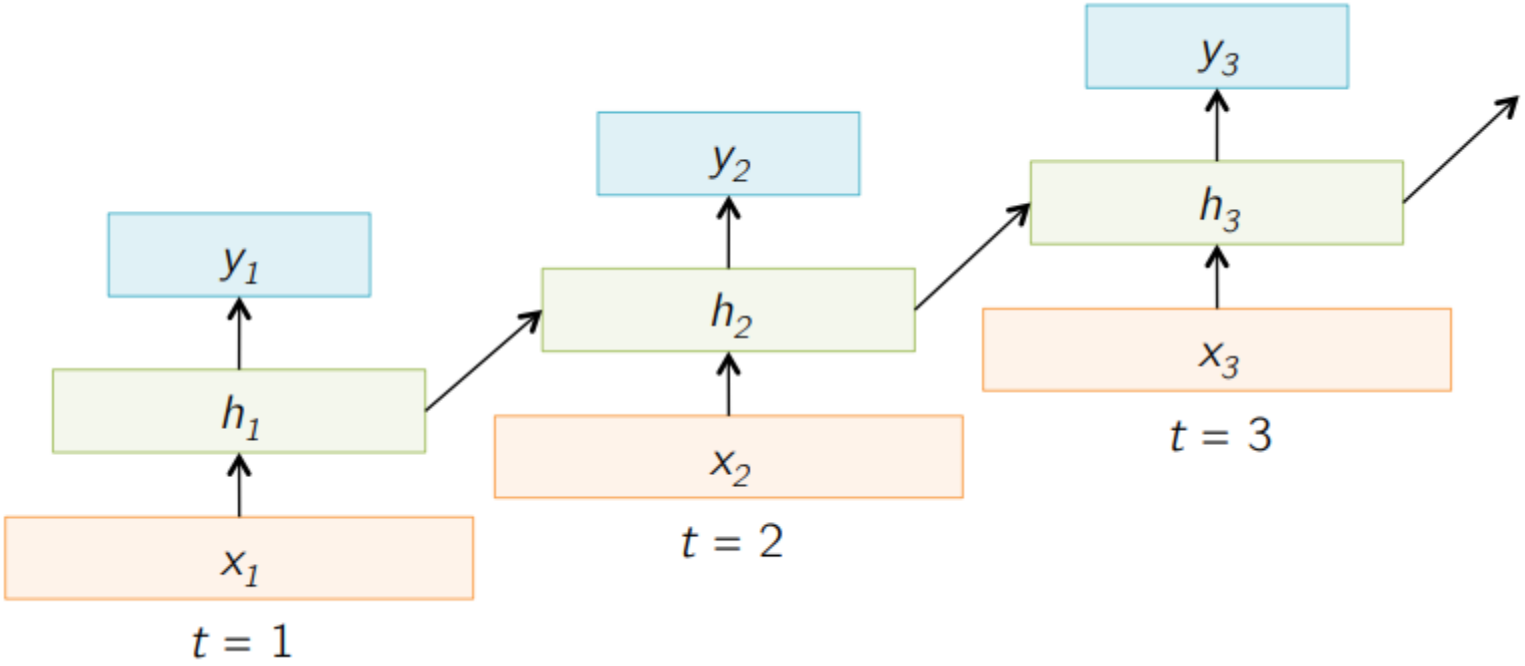
- les réseaux de neurones récurrents (en anglais *recurrent neural networks*), aussi appelés RNNs, sont une classe de réseaux de neurones qui permettent aux prédictions antérieures d'être utilisées comme entrées, par le biais d'états cachés (en anglais *hidden states*).



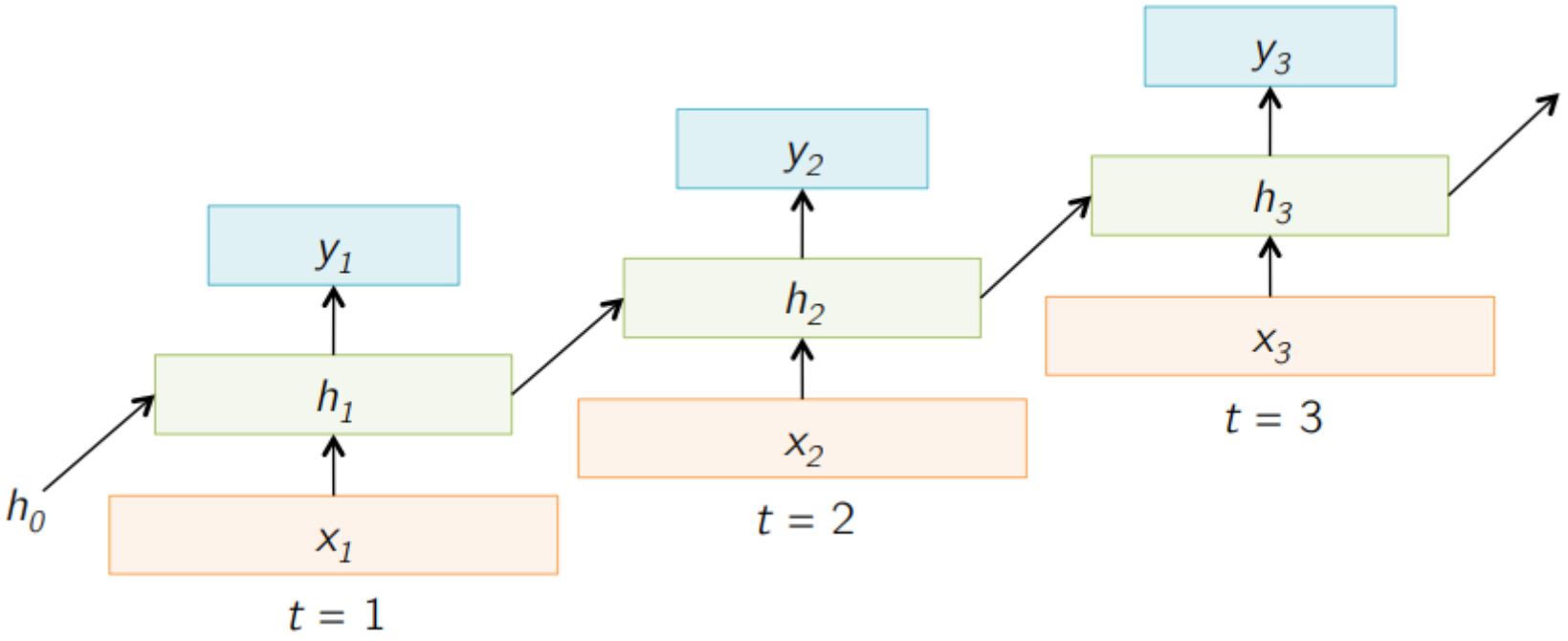
RNN



Simple RNN



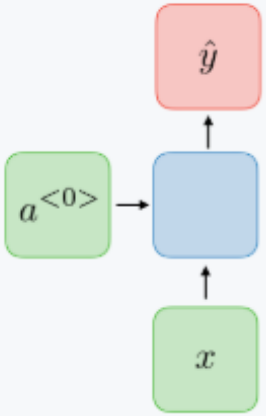
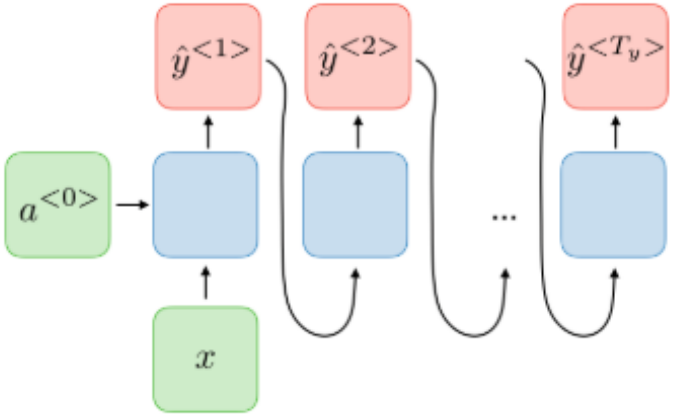
RNN



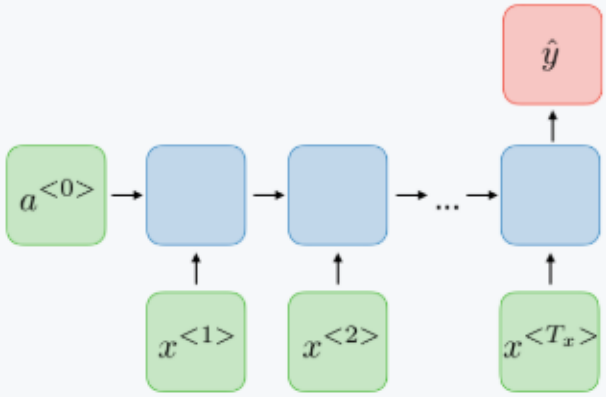
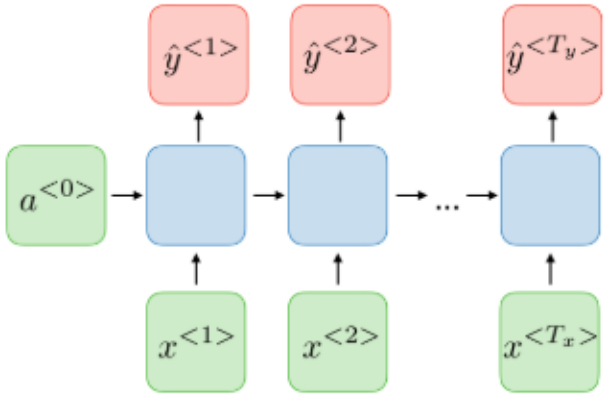
Les avantages et inconvénients des architectures de RNN traditionnelles :

Avantages	Inconvénients
• Possibilité de prendre en compte des entrées de toute taille • La taille du modèle n'augmente pas avec la taille de l'entrée • Les calculs prennent en compte les informations antérieures • Les coefficients sont indépendants du temps	• Le temps de calcul est long • **Difficulté d'accéder à des informations d'un passé lointain** • Impossibilité de prendre en compte des informations futures un état donné

Les Types

Type de RNN	Illustration	Exemple
Un à un $T_x = T_y = 1$		Réseau de neurones traditionnel
Un à plusieurs $T_x = 1, T_y > 1$		Génération de musique

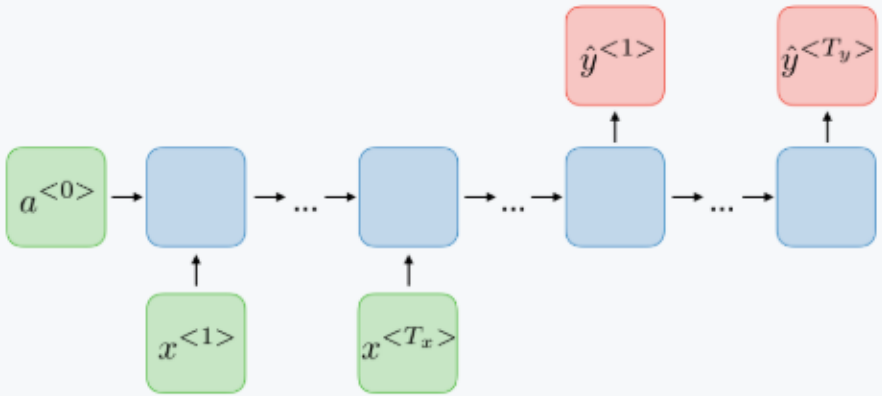
Types

Plusieurs à un $T_x > 1, T_y = 1$		Classification de sentiment
Plusieurs à plusieurs $T_x = T_y$		Reconnaissance d'entité

Types

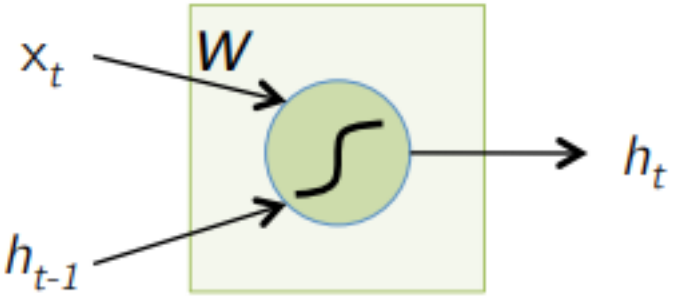
Plusieurs à plusieurs

$T_x \neq T_y$



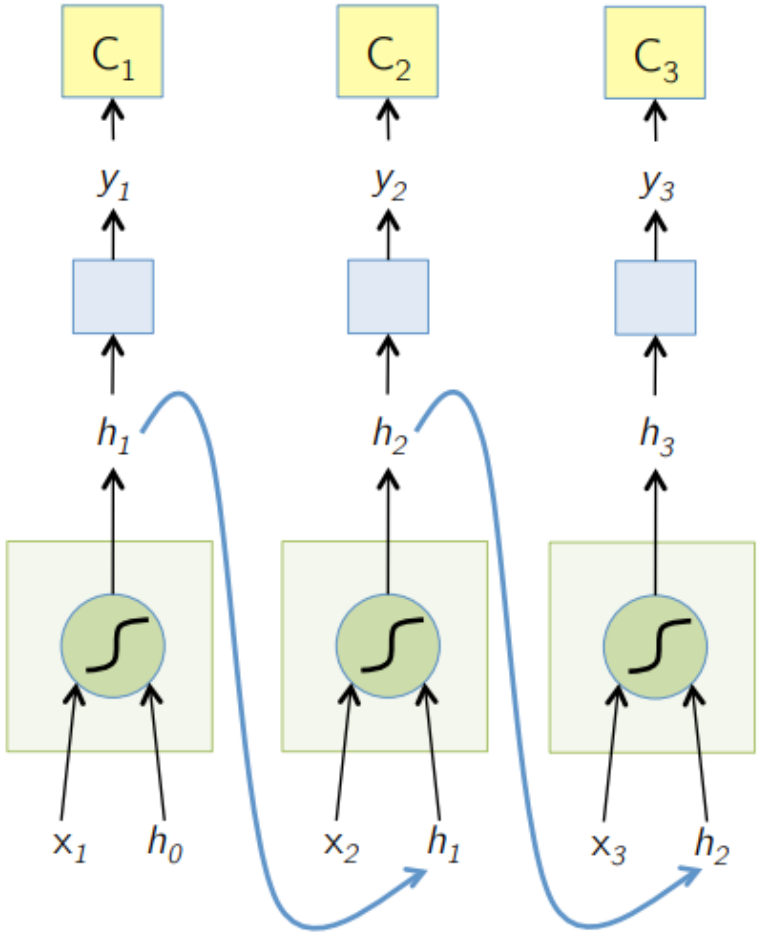
Traduction machine

The Vanilla RNN Cell



$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

The Vanilla RNN Forward

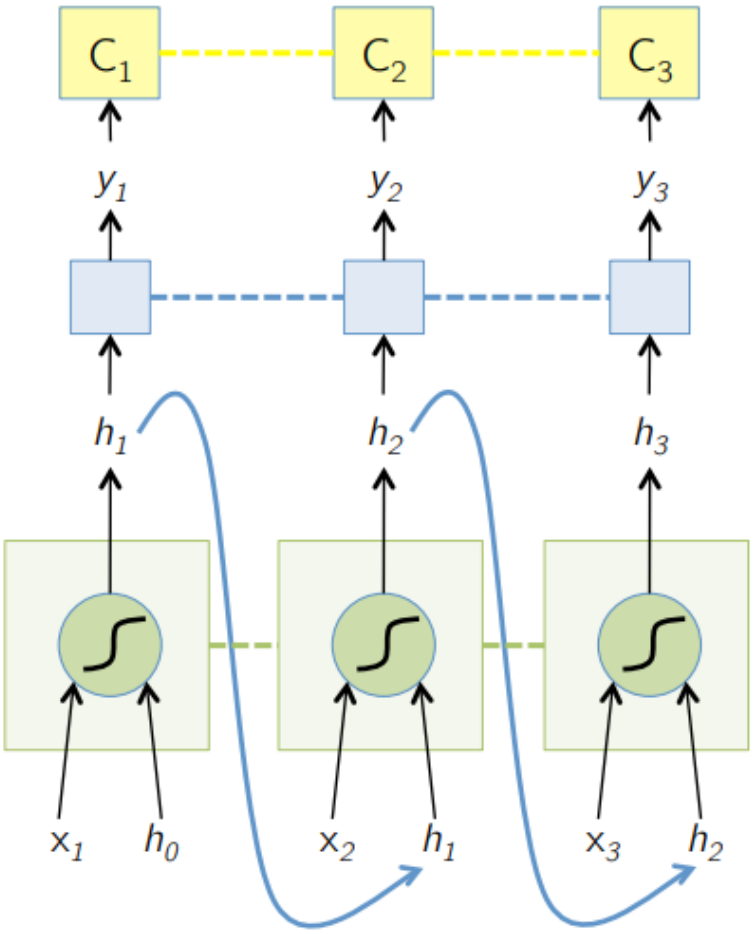


$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

$$y_t = \mathrm{F}(h_t)$$

$$C_t = \mathrm{Loss}(y_t, \mathrm{GT}_t)$$

The Vanilla RNN Forward



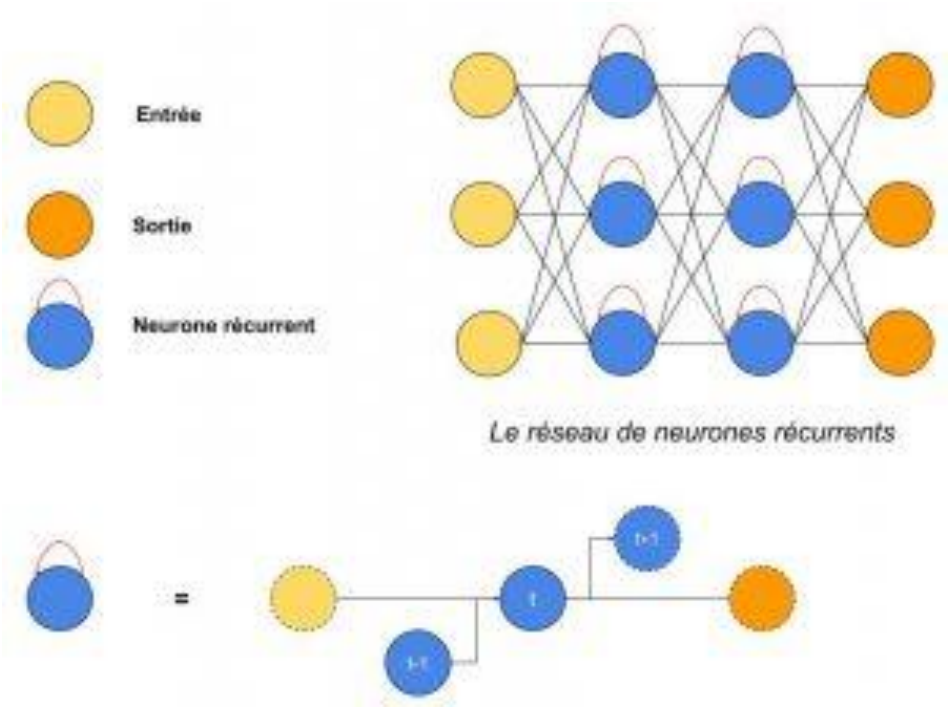
$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

$$y_t = \mathrm{F}(h_t)$$

$$C_t = \mathrm{Loss}(y_t, \mathrm{GT}_t)$$

------ indicates shared weights

Explication



Le réseau de neurones récurrents

sur chaque neurone bleu, on a une boucle

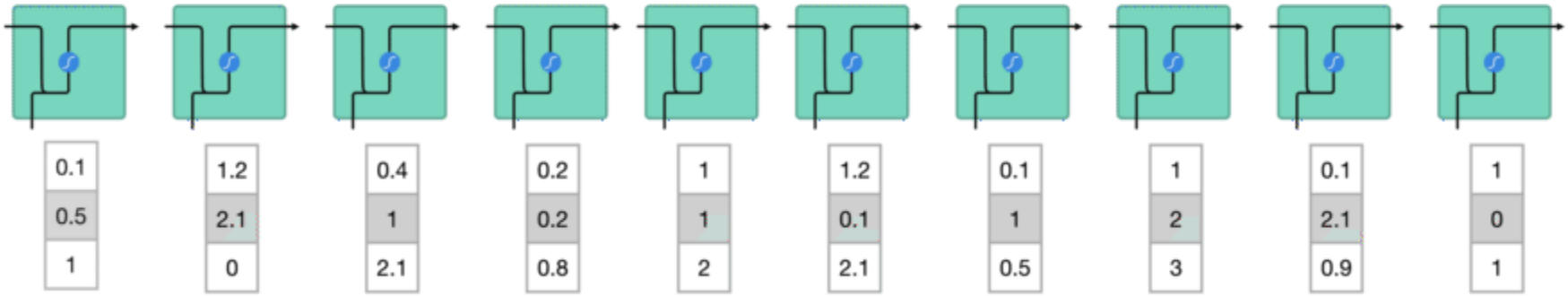
- on a une donnée t (3 valeurs, une pour chaque neurone jaune) qui arrive dans le réseau
- celles-ci *se propagent* dans le réseau comme dans un ANN
- sauf que chaque neurone bleu, en plus de recevoir les sorties pondérées des neurones précédents, reçoit également *la valeur qui sortait de lui-même pour la donnée t-1*
- la valeur de sortie de chaque neurone bleu est conservée et servira pour la donnée t+1
- On a donc une **mémoire de 1 itération** pour les neurones bleus.

Exemple

- Analyse de la phrase « Hier soir j'ai mangé un hamburger et des » et prédire le mot suivant (« frites »).

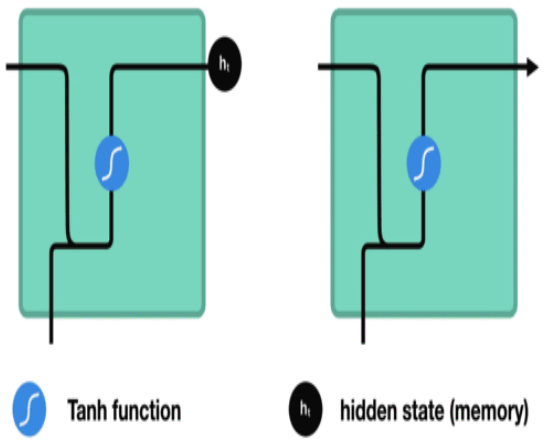
- Tout d'abord, chaque mot est transformé en un vecteur. Par exemple, « hier » devient (0.1, 0.5, 1) : cette transformation est une étape de préparation des données indispensable.
- Chaque vecteur est ensuite envoyé à tour de rôle dans le RNN, représenté ci-dessous sous la forme d'un carré vert (qui représente sa cellule i.e. un neurone).

Exemple



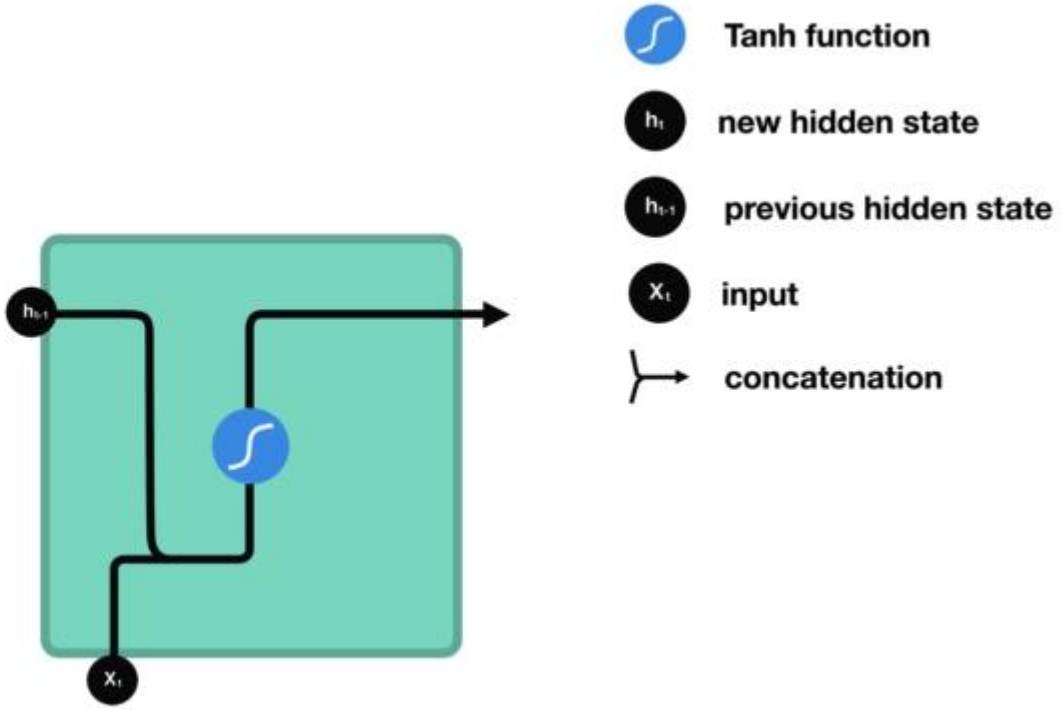
La phrase « Hier soir j'ai mangé un hamburger et des » est convertie en vecteurs puis alimente le RNN

Exemple



- Entre chaque mot envoyé dans le RNN, la sortie, notée h_t pour le mot précédent est ajoutée en entrée avec le nouveau mot.
- Cette sortie est appelée l' »état caché » (hidden state)

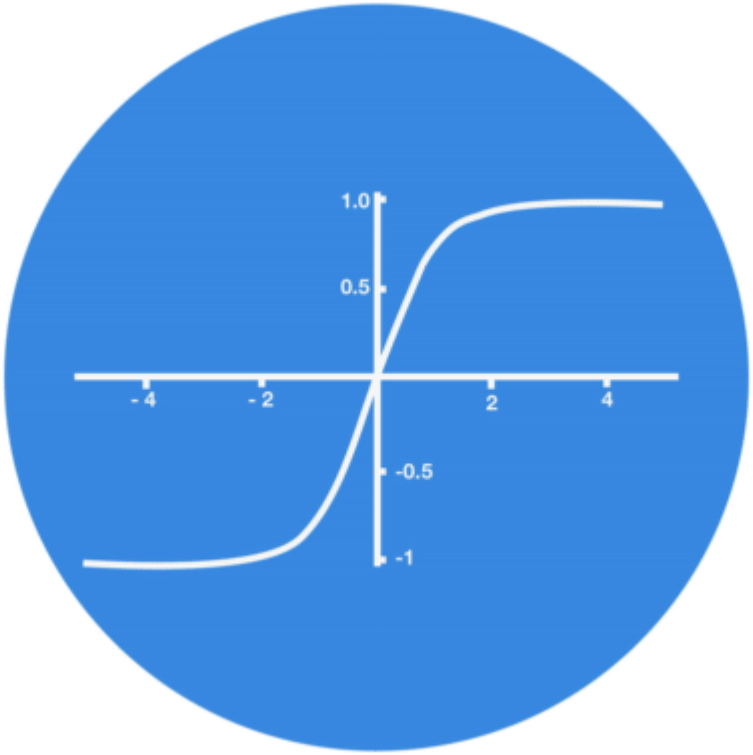
Exemple



- Le vecteur du mot courant, noté xt, et la sortie (qui est un vecteur aussi) pour le mot précédent, notée ht−1, sont mises bout-à-bout puis passent dans la fonction mathématique « tanh » avant de devenir la nouvelle sortie notée ht

Exemple

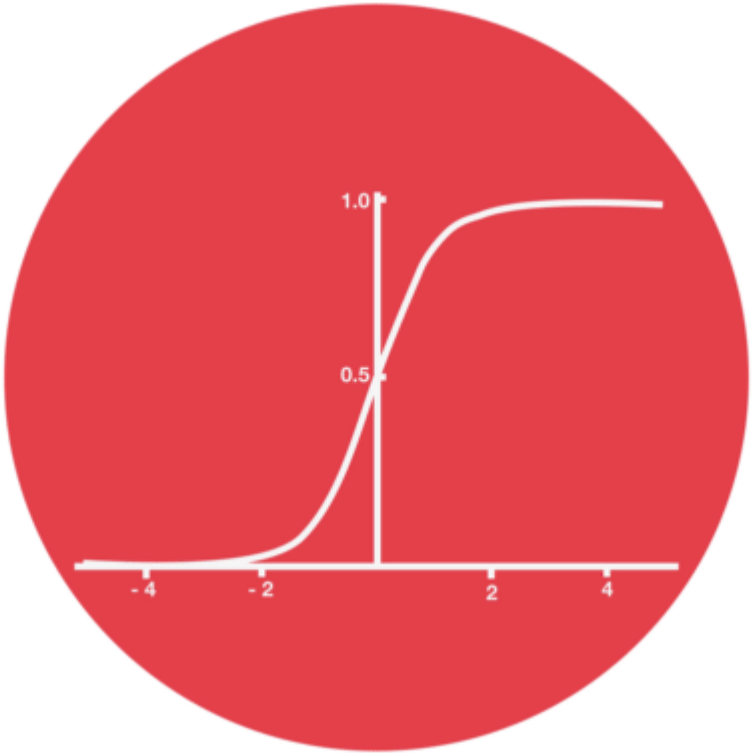
5
0.1
-0.5



- La fonction « tanh » prend une valeur en entrée et renvoie une valeur entre -1 et 1. Si on a un vecteur en entrée, alors on renvoie tanh appliquée à chaque coordonnée.

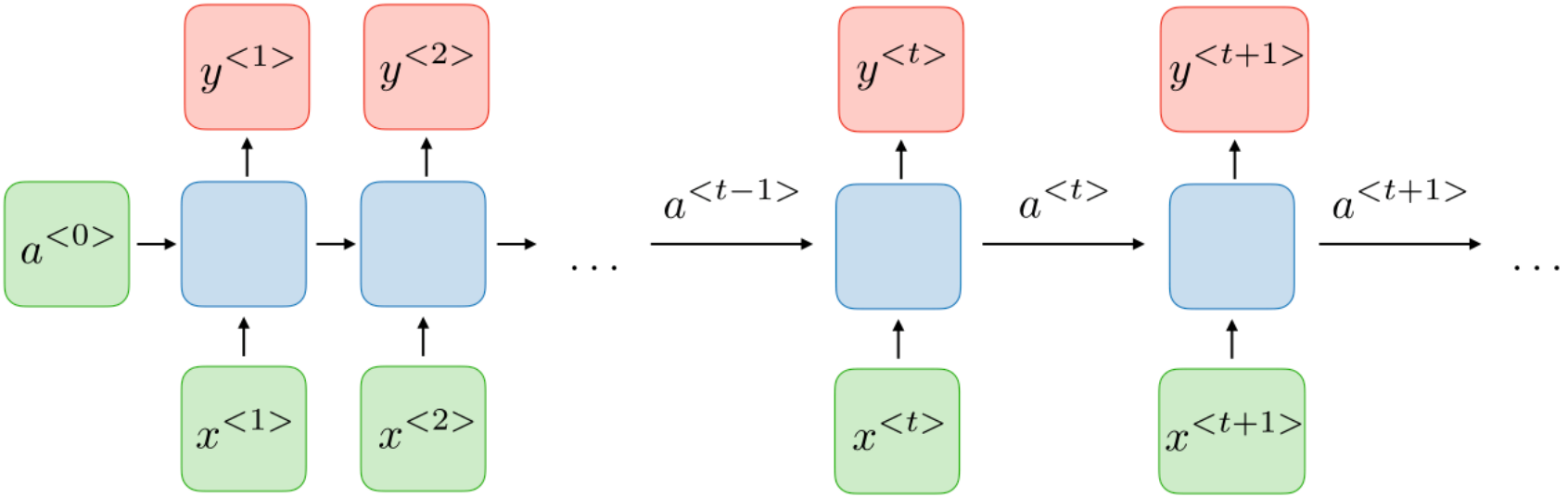
Exemple

5
0.1
-0.5



- La fonction « sigmoid »

ARCHITECTURE

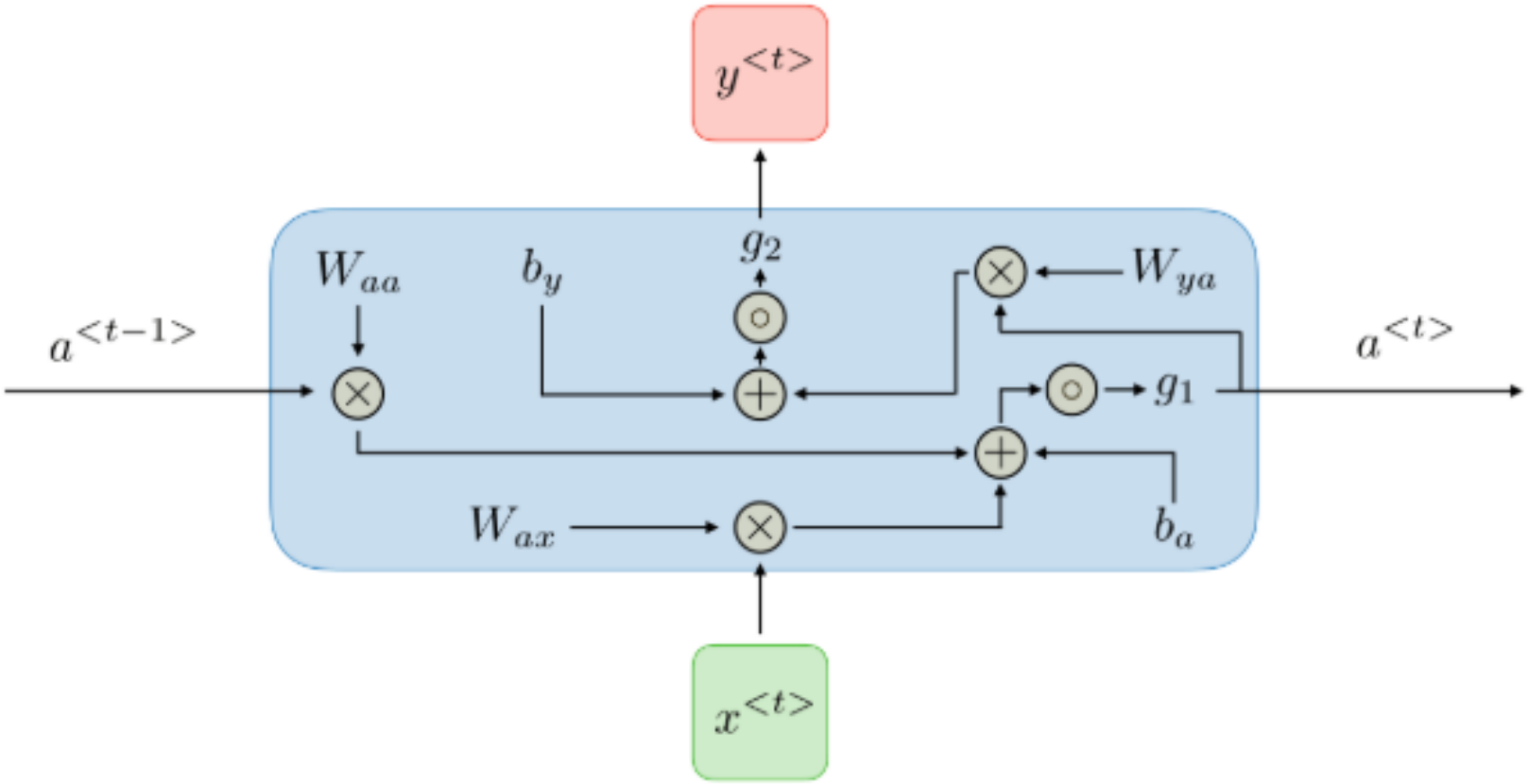


À l'instant t, l'activation $a^{<t>}$ et la sortie $y^{<t>}$ sont de la forme suivante :

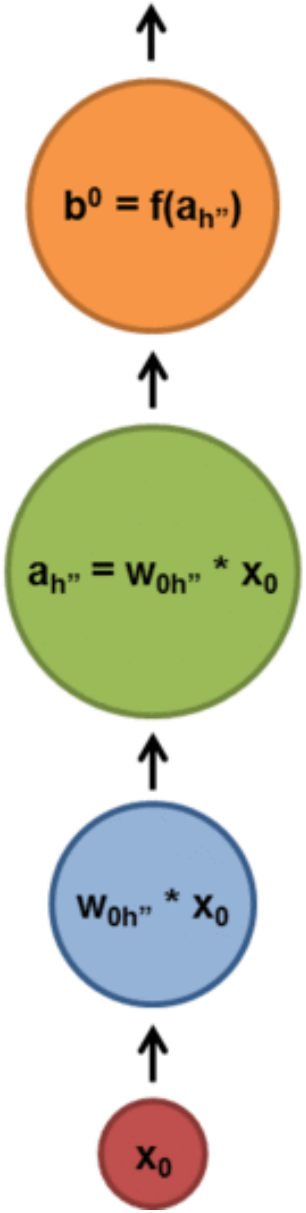
$$\boxed{a^{<t>} = g_1(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a)} \quad \text{et} \quad \boxed{y^{<t>} = g_2(W_{ya}a^{<t>} + b_y)}$$

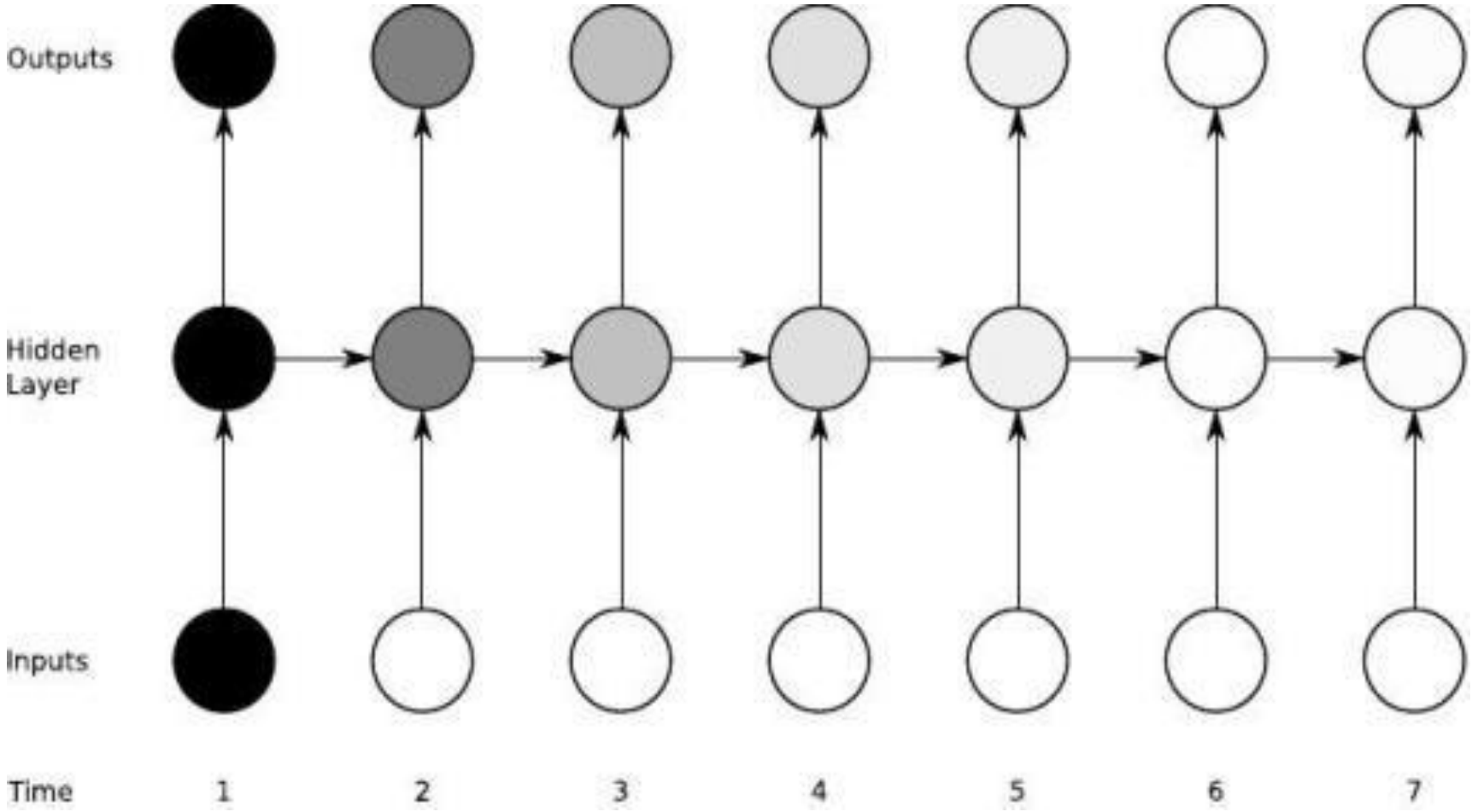
où $W_{ax}, W_{aa}, W_{ya}, b_a, b_y$ sont des coefficients indépendants du temps et où g_1, g_2 sont des fonctions d'activation.

ARCHITECTURE



b^0 is fed to next layer




Outputs
Hidden Layer
Inputs
Time
1
2
3
4
5
6
7

1.Vanilla Forward Pass

1. The forward pass of a vanilla RNN
   1. The same as that of an MLP with a single hidden layer
   2. Except that activations arrive at the hidden layer from both the current external input and the hidden layer activations one step back in time.

2. For the input to hidden units we have

$$a_h^t = \sum_{i=1}^{I} w_{ih} x_i^t + \sum_{h'=1}^{H} w_{h'h} b_{h'}^{t-1} \qquad b_h^t = \theta_h(a_h^t)$$

3. For the output unit we have

$$a_k^t = \sum_{h=1}^{H} w_{hk} b_h^t$$

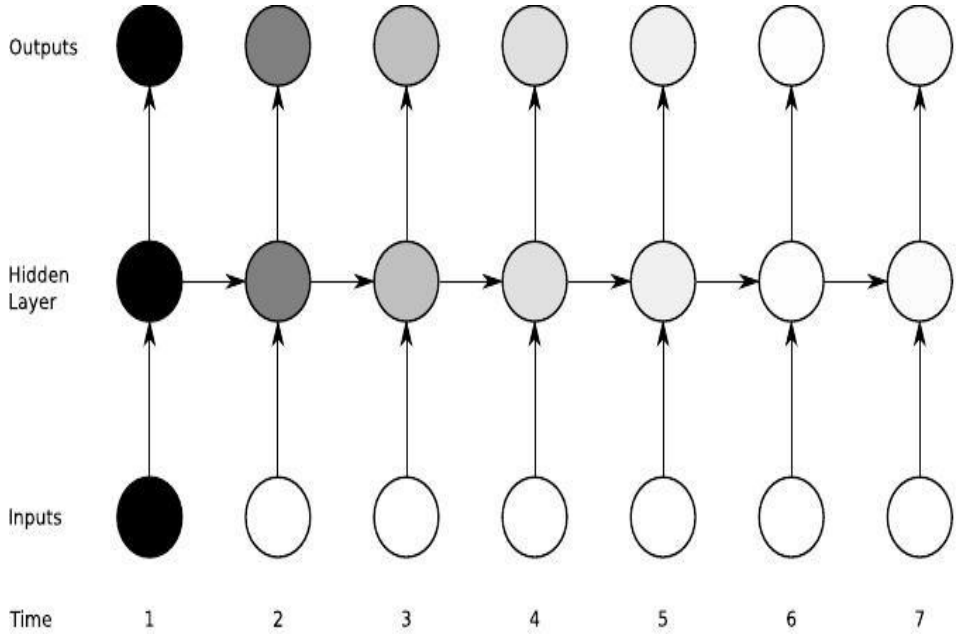
Vanilla Forward Pass

1. The complete sequence of hidden activations can be calculated by starting at t = 1 and recursively applying the three equations, incrementing t at each step.

$$a_h^t = \sum_{i=1}^{I} w_{ih} x_i^t + \sum_{h'=1}^{H} w_{h'h} b_{h'}^{t-1}$$

$$b_h^t = \theta_h(a_h^t)$$

$$a_k^t = \sum_{h=1}^{H} w_{hk} b_h^t$$



Fonction de loss

- Dans le contexte des réseaux de neurones récurrents, la fonction de loss $\mathcal{L}$ prend en compte le loss à chaque temps T de la manière suivante :
-

$$\mathcal{L}(\hat{y}, y) = \sum_{t=1}^{T_y} \mathcal{L}(\hat{y}^{<t>}, y^{<t>})$$

Rétro-propagation des erreurs

- la rétro-propagation dans le temps (Back-Propagation Through Time - BPTT)
- L'algorithme de BPTT consiste à appliquer récursivement les règles de dérivation des fonctions composées pour tous les pas de temps en parcourant le temps à rebours ($t : t_f \longrightarrow 1$).

- **Les dangers de la rétro propagation à travers le temps : disparition et explosion du gradient**

2.Vanilla Backward Pass

1. Given the partial derivatives of the objective function with respect to the network outputs, we now need the derivatives with respect to the weights.
2. We focus on BPTT since it is both conceptually simpler and more efficient in computation time (though not in memory). Like standard back-propagation, BPTT consists of a repeated application of the chain rule.

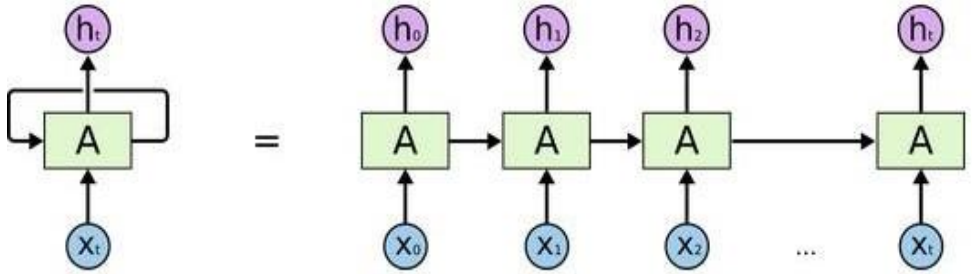
$$a_h^t = \sum_{i=1}^{I} w_{ih} x_i^t + \sum_{h'-1}^{H} w_{h'h} b_{h'}^{t-1}$$

$$b_h^t = \theta_h(a_h^t)$$

$$a_k^t = \sum_{h=1}^{H} w_{hk} b_h^t$$

2.Vanilla Backward Pass

1. Back-propagation through time
   1. . It's just the standard back-propagation.



An unrolled recurrent neural network.

$$\delta_h^t = \theta'(a_h^t)\left(\sum_{k=1}^{K}\delta_k^t w_{hk} + \sum_{h'=1}^{H}\delta_{h'}^{t+1} w_{hh'}\right),$$

$$a_h^t = \sum_{i=1}^{I} w_{ih}x_i^t + \sum_{h'=1}^{H} w_{h'h}b_{h'}^{t-1}$$

$$b_h^t = \theta_h(a_h^t)$$

$$\delta_j^t \stackrel{\text{def}}{=} \frac{\partial O}{\partial a_j^t}$$

$$a_k^t = \sum_{h=1}^{H} w_{hk}b_h^t$$

2.Vanilla Backward Pass

1. Back-propagation through time
   1. The complete sequence of delta terms can be calculated by starting at t = T and recursively applying the below functions, decrementing t at each step.
   2. Note that $\delta_j^{T+1} = 0 \, \forall j$, since no error is received from beyond the end of the sequence.

$$\delta_h^t = \theta'(a_h^t) \left(\sum_{k=1}^{K} \delta_k^t w_{hk} + \sum_{h'=1}^{H} \delta_{h'}^{t+1} w_{hh'} \right),$$

$$a_h^t = \sum_{i=1}^{I} w_{ih} x_i^t + \sum_{h'=1}^{H} w_{h'h} b_{h'}^{t-1}$$

$$b_h^t = \theta_h(a_h^t)$$

$$a_k^t = \sum_{h=1}^{H} w_{hk} b_h^t$$

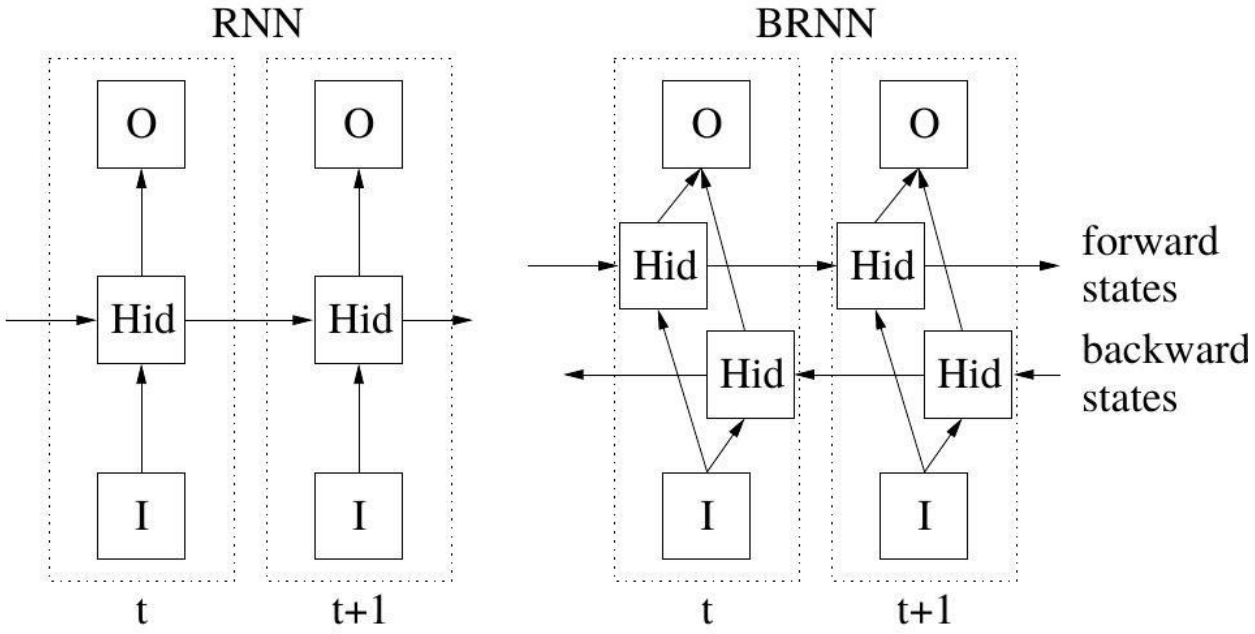
$$\delta_j^t \stackrel{\text{def}}{=} \frac{\partial O}{\partial a_j^t}$$

   3. Finally, bearing in mind that the weights to and from each unit in the hidden layer are the same at every time-step, we sum over the whole sequence to get the derivatives with respect to each of the network weights

$$\frac{\partial O}{\partial w_{ij}} = \sum_{t=1}^{T} \frac{\partial O}{\partial a_j^t} \frac{\partial a_j^t}{\partial w_{ij}} = \sum_{t=1}^{T} \delta_j^t b_i^t$$

3.Vanilla Bidirectional Pass

1. For many sequence labeling tasks, we would like to have access to future.



3.Vanilla Bidirectional Pass

1. Algorithm looks like this

for $t = 1$ to T **do**
Do forward pass for the forward hidden layer, storing activations at each timestep
for $t = T$ to 1 **do**
Do forward pass for the backward hidden layer, storing activations at each timestep
for $t = 1$ to T **do**
Do forward pass for the output layer, using the stored activations from both hidden layers

Algorithm 3.1: BRNN Forward Pass

Similarly, the backward pass proceeds as for a standard RNN trained with BPTT, except that all the output layer δ terms are calculated first, then fed back to the two hidden layers in opposite directions:

for $t = T$ to 1 **do**
Do BPTT backward pass for the output layer only, storing δ terms at each timestep
for $t = T$ to 1 **do**
Do BPTT backward pass for the forward hidden layer, using the stored δ terms from the output layer
for $t = 1$ to T **do**
Do BPTT backward pass for the backward hidden layer, using the stored δ terms from the output layer

Algorithm 3.2: BRNN Backward Pass

4.Training of Vanilla RNN

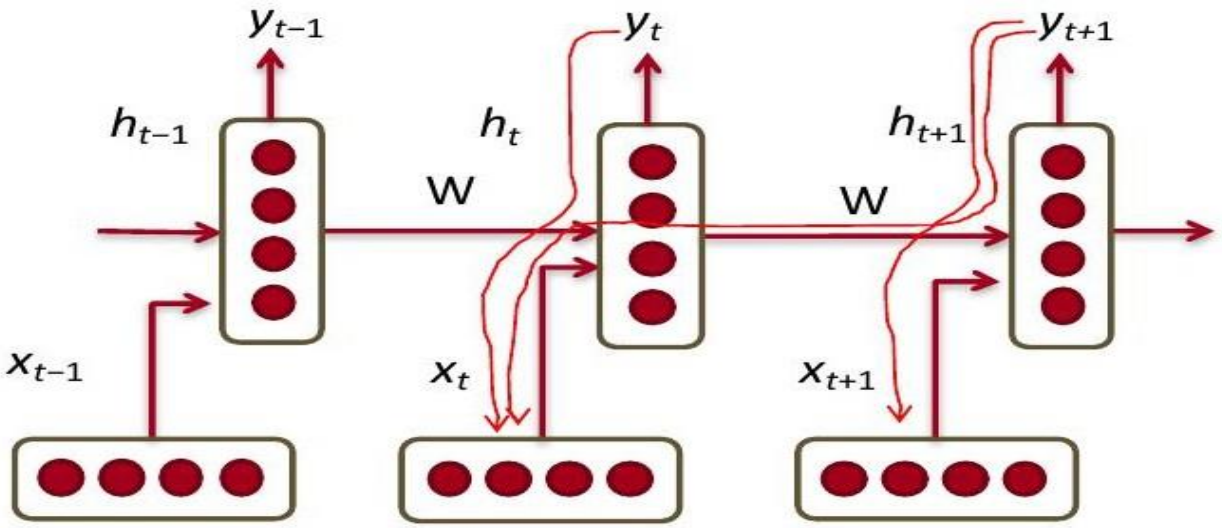
1. So far we have discussed how RNN can be differentiated with respect to suitable objective functions, and thereby they could be trained with any gradient-descent based algorithm
   1. just treat them as a normal CNN

$$\Delta \mathbf{w}(n) = -\alpha \frac{\partial O}{\partial \mathbf{w}(n)}$$

2. One of the great things about RNN: lots of engineering choices
   1. Preprocessing and postprocessing

5.Vanishing and exploding gradient problems

1. Multiply the same matrix at each time step during back-prop



5.Vanishing and exploding gradient problems

1. Toy example how gradient vanishes
   1. Similar but simpler RNN formulation:

$$h_t = Wf(h_{t-1}) + W^{(hx)}x_{[t]}$$

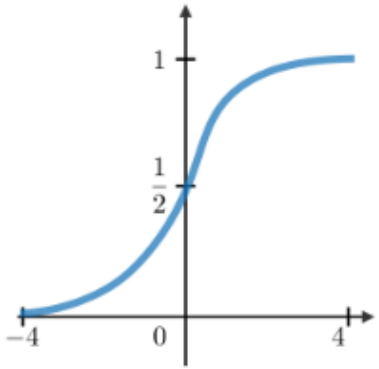
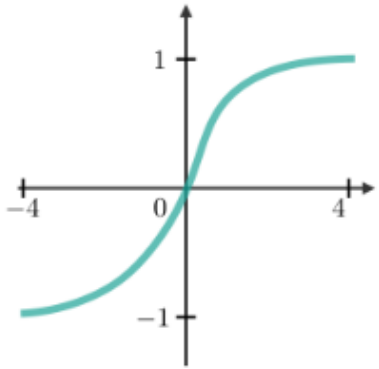
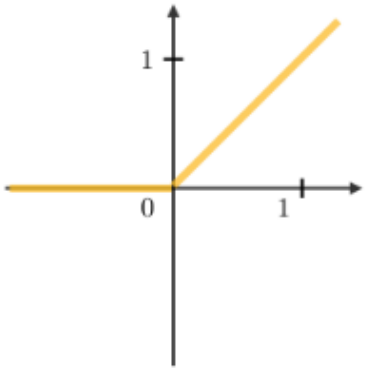
$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} = \prod_{j=k+1}^{t} W^T \mathrm{diag}[f'(h_{j-1})]$$

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| \leq \|W^T\| \|\mathrm{diag}[f'(h_{j-1})]\| \leq \beta_W \beta_h$$

$$\left\| \frac{\partial h_t}{\partial h_k} \right\| = \left\| \prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \right\| \leq (\beta_W \beta_h)^{t-k}$$

2. Solutions?
   1. For vanishing gradients: Initialization + ReLus
   2. Trick for exploding gradient: clipping trick

Activation

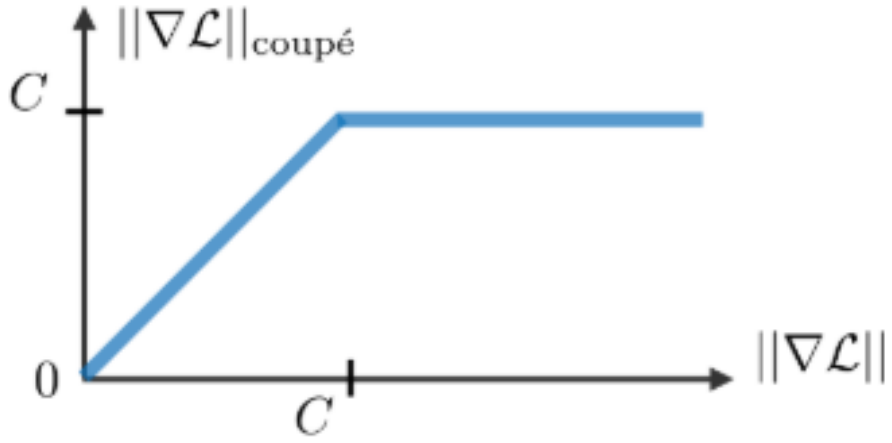
Sigmoïde	Tanh	RELU
$g(z) = \frac{1}{1+e^{-z}}$	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g(z) = \max(0, z)$
		

vanishing gradient and *exploding gradient*

- **Gradient qui disparait/explose** — Les phénomènes de gradient qui disparait et qui explose (en anglais *vanishing gradient* et *exploding gradient*) sont souvent rencontrés dans le contexte des RNNs. Ceci est dû au fait qu'il est difficile de capturer des dépendances à long terme à cause du gradient multiplicatif qui peut décroître/croître de manière exponentielle en fonction du nombre de couches.

Coupure de gradient

- Cette technique est utilisée pour atténuer le phénomène de gradient qui explose qui peut être rencontré lors de l'étape de backpropagation. En plafonnant la valeur qui peut être prise par le gradient, ce phénomène est maîtrisé en pratique.



Types de porte

- Pour remédier au problème du gradient qui disparait, certains types de porte sont spécifiquement utilisés dans des variantes de RNNs et ont un but bien défini. Les portes sont souvent notées Γ et sont telles que :

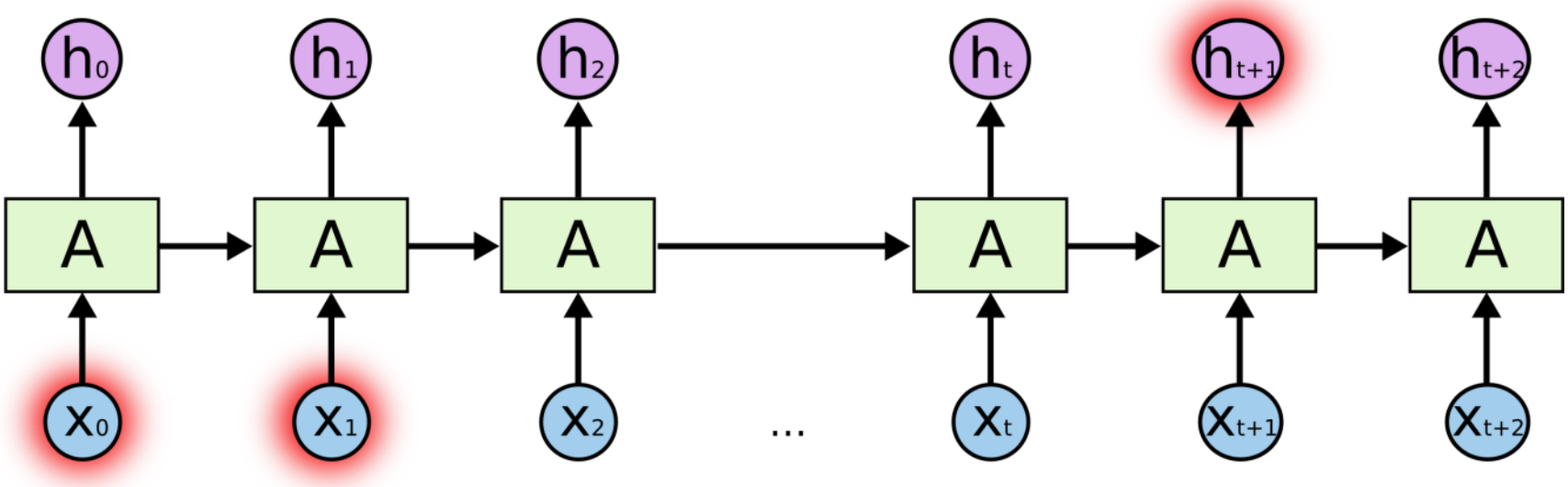
$$\Gamma = \sigma(W x^{<t>} + U a^{<t-1>} + b)$$

- où $\mathcal{W}, \mathcal{U}, \mathcal{b}$ sont des coefficients spécifiques à la porte et σ est une sigmoïde.

Type de porte

Type de porte	Rôle	Utilisée dans
Porte d'actualisation Γ_u	Dans quelle mesure le passé devrait être important ?	GRU, LSTM
Porte de pertinence Γ_r	Enlever les informations précédentes ?	GRU, LSTM
Porte d'oubli Γ_f	Enlever une cellule ?	LSTM
Porte de sortie Γ_o	Combien devrait-on révéler d'une cellule ?	LSTM

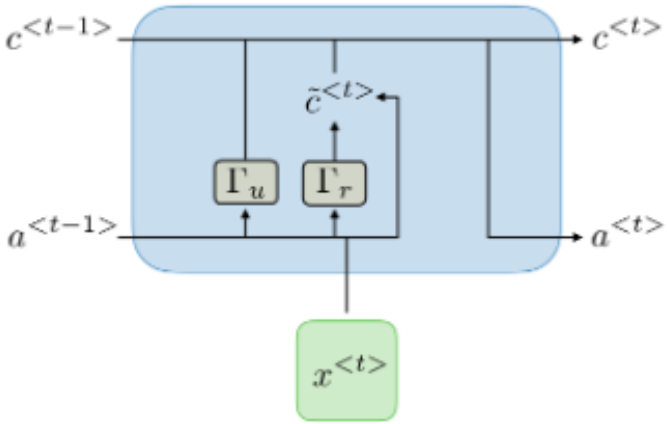
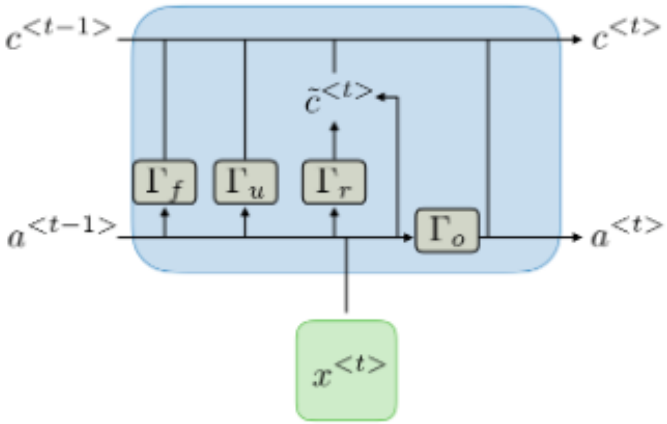
Mémoire



GRU/LSTM

- Les unités de porte récurrente (en anglais *Gated Recurrent Unit*) (GRU) et les unités de mémoire à long/court terme (en anglais *Long Short-Term Memory units*) (LSTM) apaisent le problème du gradient qui disparait rencontré par les RNNs traditionnels, où le LSTM peut être vu comme étant une généralisation du GRU.

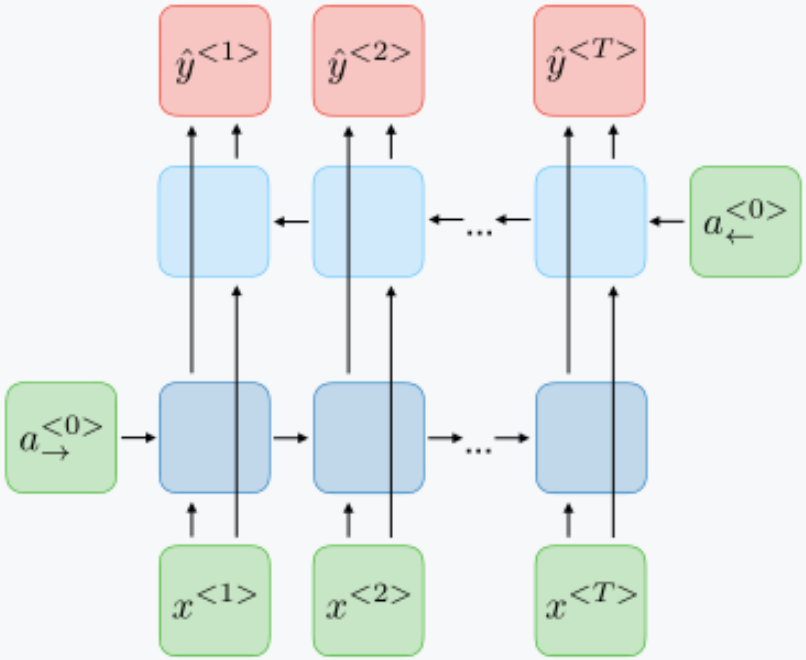
Equations

Caractérisation	**Gated Recurrent Unit** (GRU)	**Long Short-Term Memory** (LSTM)
$\tilde{c}^{<t>}$	$\tanh(W_c[\Gamma_r \star a^{<t-1>}, x^{<t>}] + b_c)$	$\tanh(W_c[\Gamma_r \star a^{<t-1>}, x^{<t>}] + b_c)$
$c^{<t>}$	$\Gamma_u \star \tilde{c}^{<t>} + (1 - \Gamma_u) \star c^{<t-1>}$	$\Gamma_u \star \tilde{c}^{<t>} + \Gamma_f \star c^{<t-1>}$
$a^{<t>}$	$c^{<t>}$	$\Gamma_o \star c^{<t>}$
Dépendances		

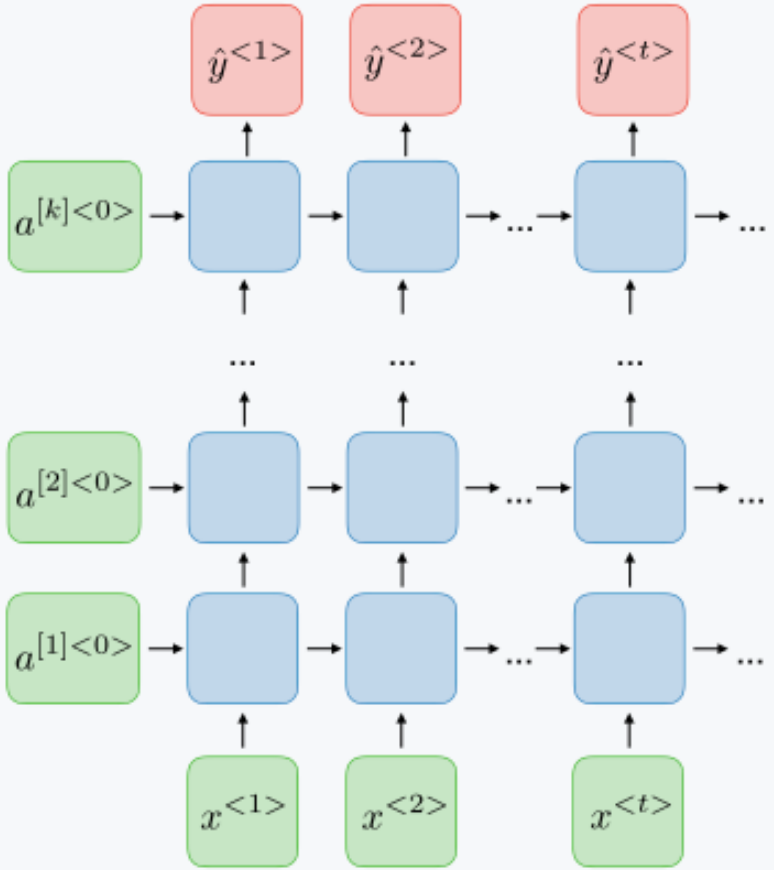
Remarque : le signe ⋆ dénote le produit de Hadamard entre deux vecteurs.

Variantes de RNNs

Bi-directionnel (BRNN)



Profond (DRNN)



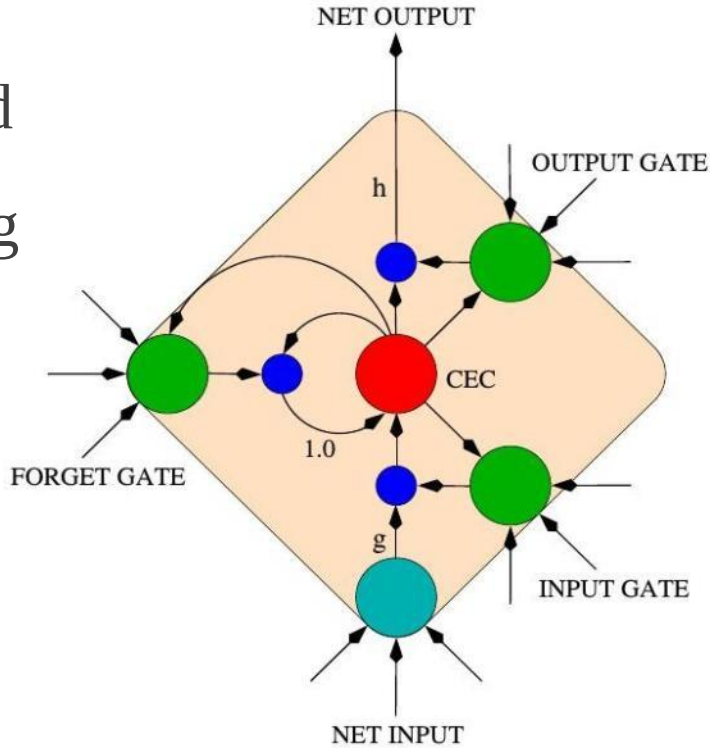
LSTM

Definition

1. As discussed earlier, for standard RNN architectures, the range of context that can be accessed is limited.
    1. The problem is that the influence of a given input on the hidden layer, and therefore on the network output, either decays or blows up exponentially as it cycles around the network's recurrent connections.
1. The most effective solution so far is the Long Short Term Memory (LSTM) architecture (Hochreiter and Schmidhuber, 1997).
2. The LSTM architecture consists of a set of recurrently connected subnets, known as memory blocks. These blocks can be thought of as a differentiable version of the memory chips in a digital computer. Each block contains one or more self-connected memory cells and three multiplicative units that provide continuous analogues of write, read and reset operations for the cells
    1. The input, output and forget gates.

Definition

1. The multiplicative gates allow LSTM memory cells to store and access information over long periods of time, thereby avoiding the vanishing gradient problem
    1. For example, as long as the input gate remains closed (i.e. has an activation close to 0), the activation of the cell will not be overwritten by the new inputs arriving in the network, and can therefore be made available to the net much later in the sequence, by opening the output gate.



Definition

1. Comparison

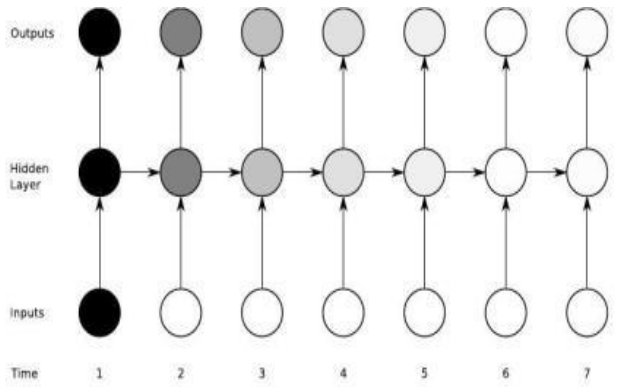


Figure 4.1: **Vanishing gradient problem for RNNs.** The shading of the nodes indicates the sensitivity over time of the network nodes to the input at time one (the darker the shade, the greater the sensitivity). The sensitivity decays exponentially over time as new inputs overwrite the activation of hidden unit and the network 'forgets' the first input.

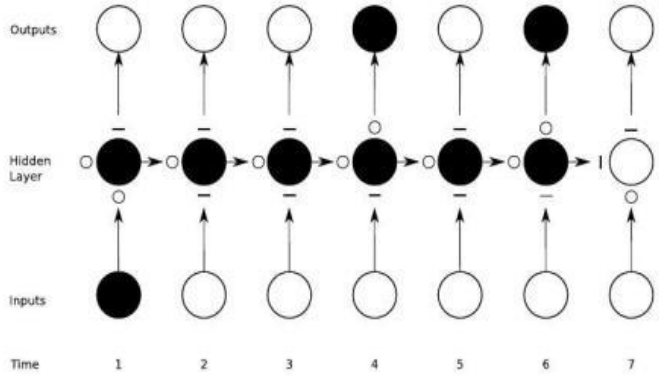
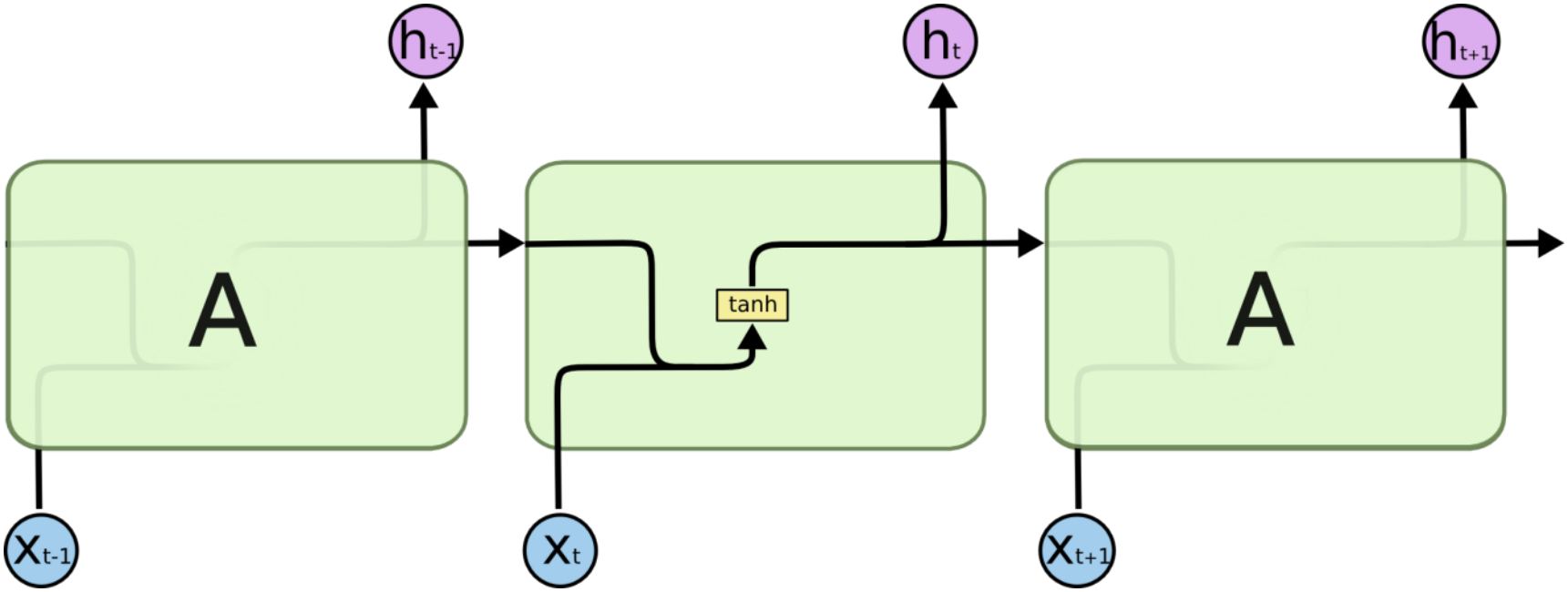


Figure 4.3: **Preservation of gradient information by LSTM.** As in Figure 4.1 the shading of the nodes indicates their sensitivity to the input unit at time one. The state of the input, forget, and output gate states are displayed below, to the left and above the hidden layer node, which corresponds to a single memory cell. For simplicity, the gates are either entirely open ('O') or closed ('—'). The memory cell 'remembers' the first input as long as the forget gate is open and the input gate is closed, and the sensitivity of the output layer can be switched on and off by the output gate without affecting the cell.

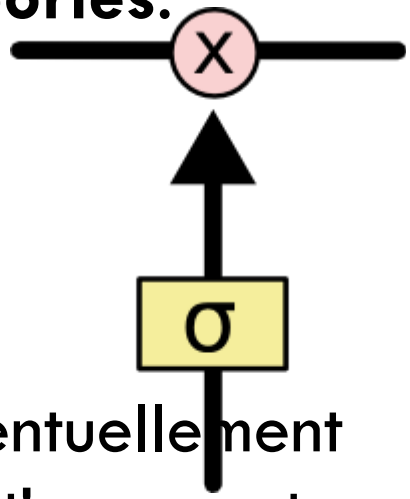
LSTM



The repeating **module in a standard RNN contains a single layer.**

Portes

- Le LSTM a la capacité de supprimer ou d'ajouter des informations à l'état de la cellule, soigneusement régulées par des structures appelées **portes.**



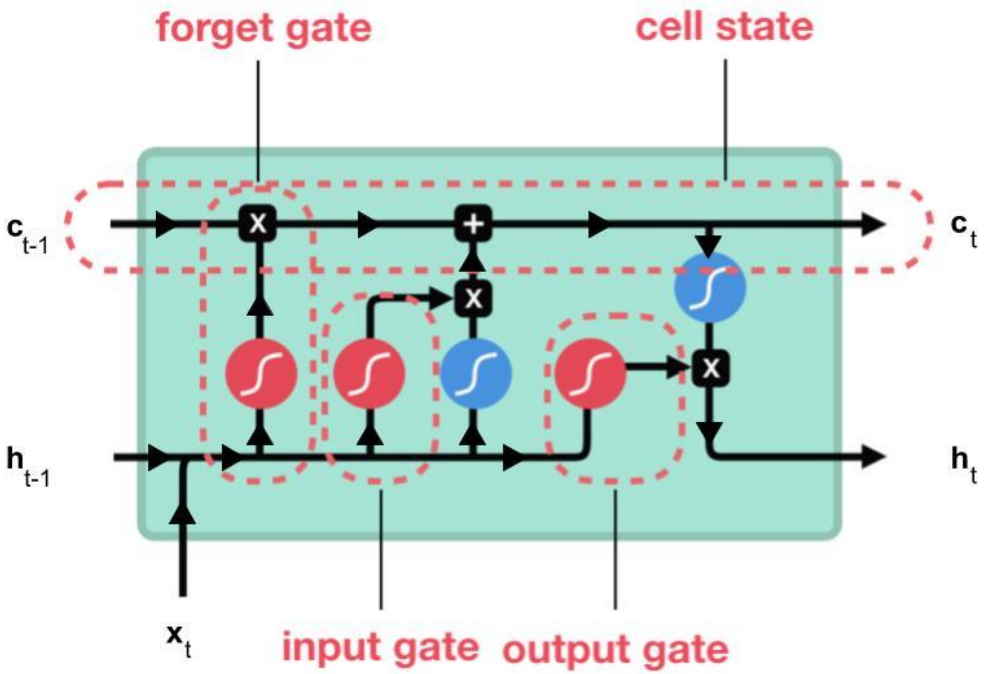
- Les portes sont un moyen de laisser éventuellement passer l'information. Ils sont composés d'une couche de réseau neural sigmoïde et d'une opération de multiplication ponctuelle.

Sigmoïde

- La couche sigmoïde génère des nombres compris entre zéro et un, décrivant la quantité de chaque composant à laisser passer. Une valeur de zéro signifie "ne laisse rien passer", alors qu'une valeur de un signifie "laisse tout passer!"

- Un LSTM a trois de ces portes, pour protéger et contrôler l'état de la cellule.

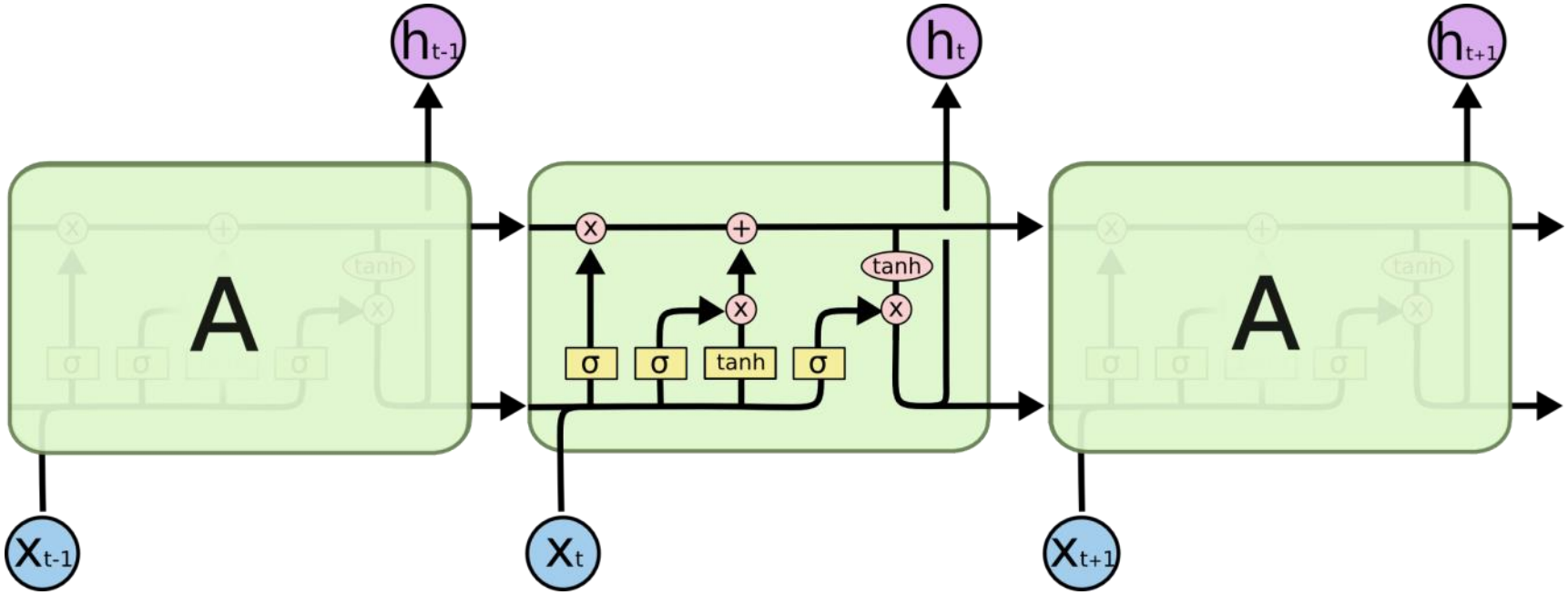
LSTM



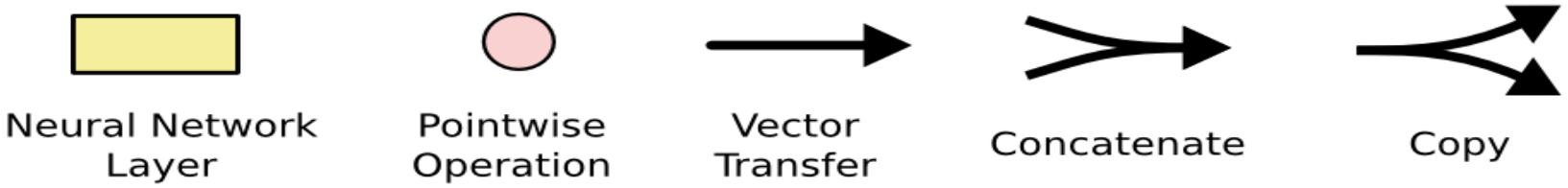
- **LSTM**, qui signifie *Long Short-Term Memory*, est une cellule composée de trois « portes »:
- ce sont des zones de calculs qui régulent le flot d'informations (en réalisant des actions spécifiques).
- On a également deux types de sorties (nommées états).

1. Forget gate (porte d'oubli)
2. Input gate (porte d'entrée)
3. Output gate (porte de sortie)
4. Hidden state (état caché)
5. Cell state (état de la cellule)

interacting layers



The repeating module in an LSTM contains four interacting layers.



Opérations internes que l'on retrouve dans le LSTM



Sigmoïde (notée s)
Fonction à appliquer



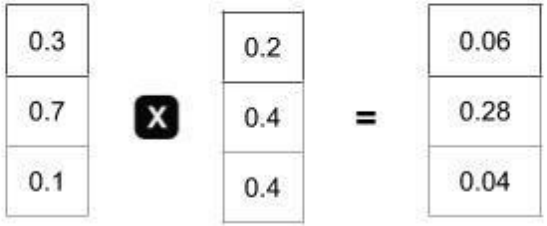


Tanh (notée th)
Fonction à appliquer





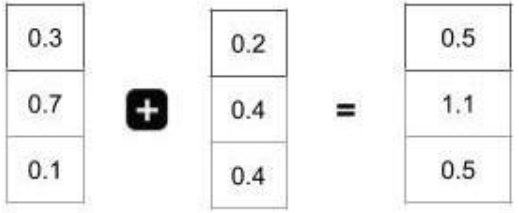
Multiplication
Multiplier les coordonnées face à face

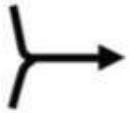


Opérations internes que l'on retrouve dans le LSTM

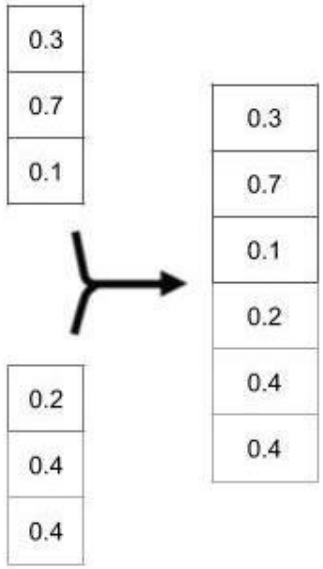


Addition
Additionner les coordonnées face à face





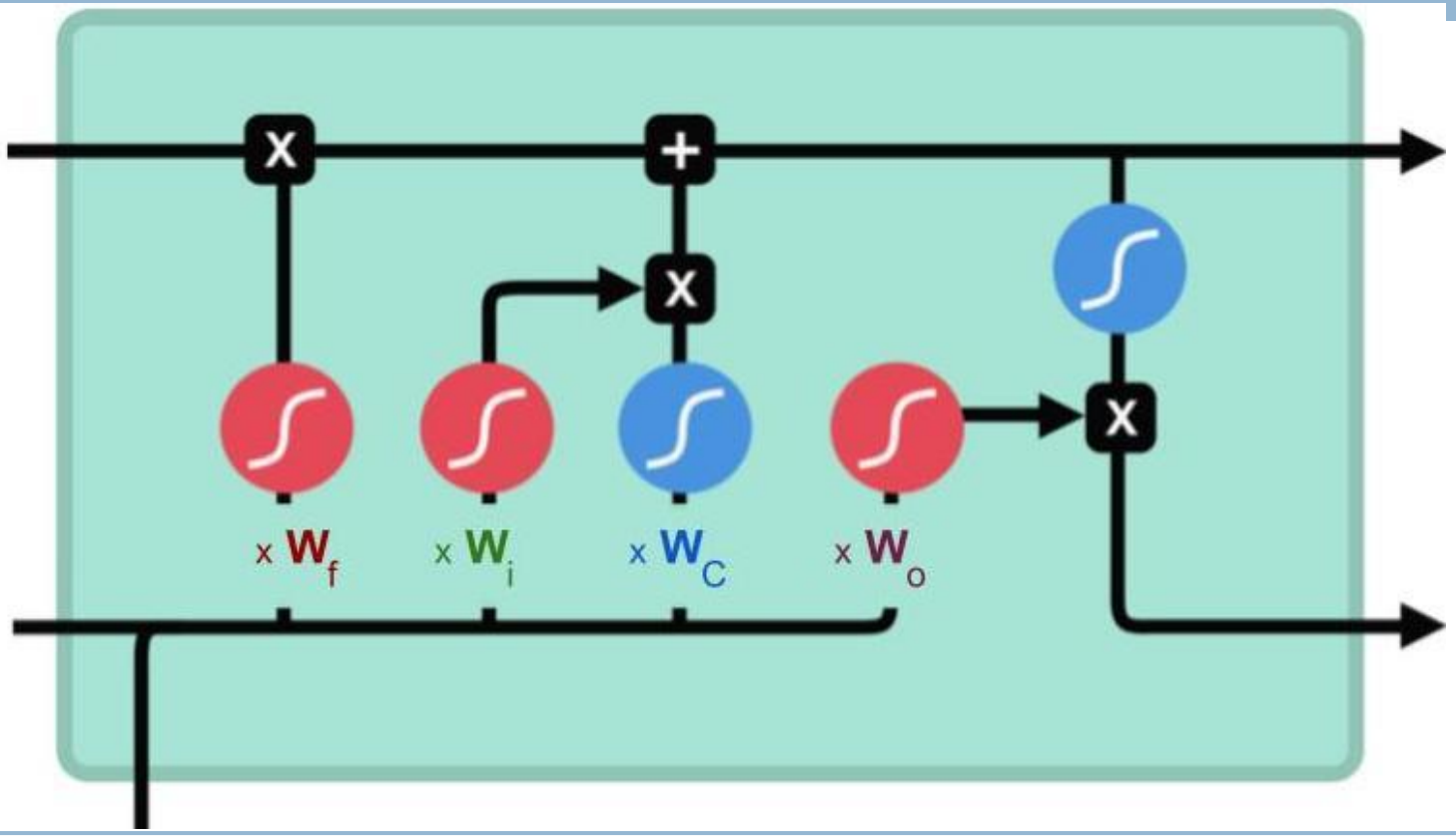
Concaténation
Mettre les deux vecteurs bout-à-bout



Modèle

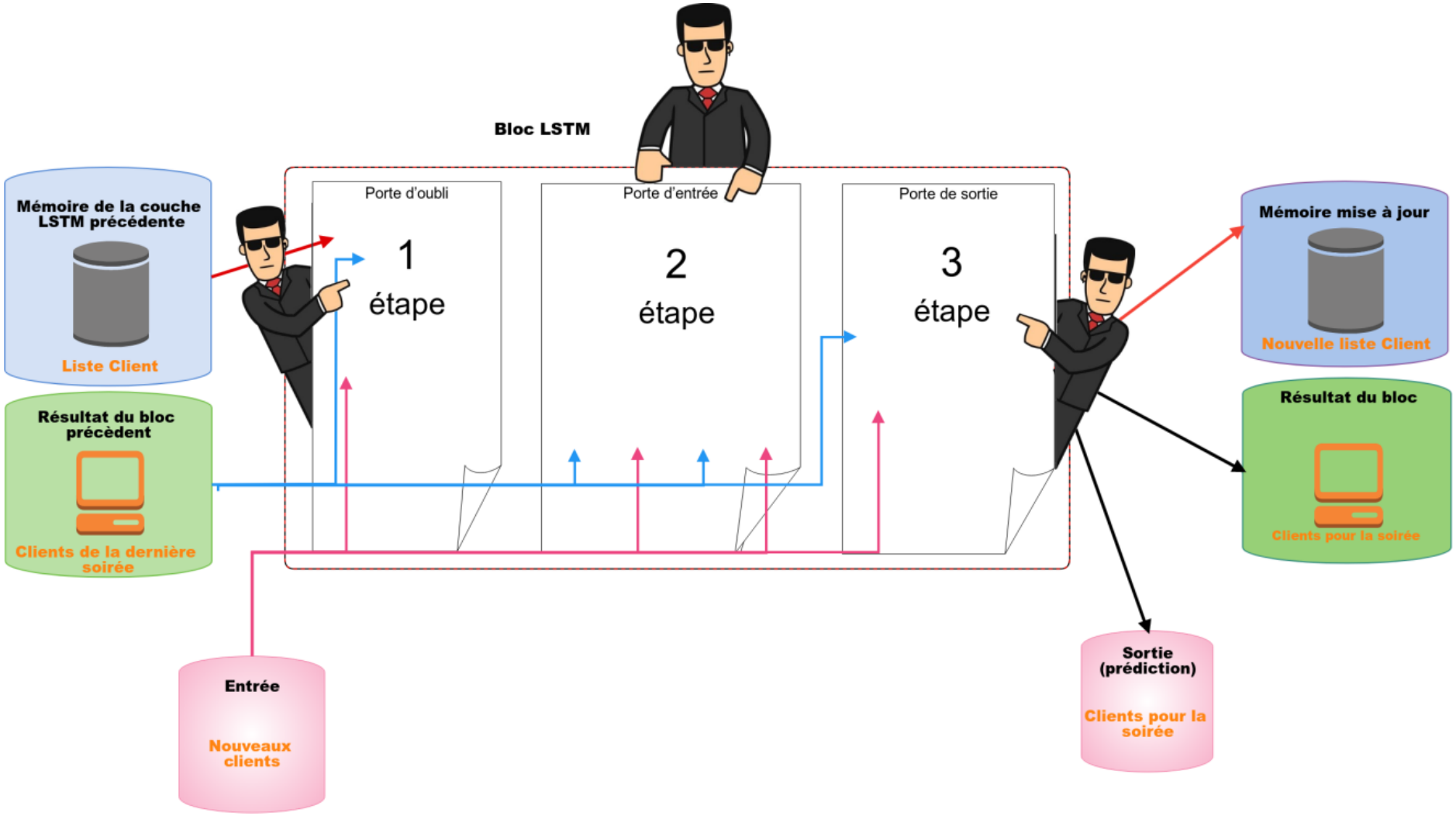
- Une couche de LSTM L″¡ est composée de quatre couches RNN qui interagissent entre elles.
- Trois de ces couches RNN ont pour fonction de transfert la fonction logistique (et donc des sorties entre 0 et 1) et agissent comme des portes logiques contrôlant :
  - la quantité d'information qui entre dans les cellules LSTM de la couche ¡,
  - la quantité d'information qui est retenue par l'état interne des cellules d'un pas sur l'autre,
  - la quantité d'information qui sort des cellules LSTM de la couche ¡
- La dernière des quatre couches fonctionne comme une couche RNN standard et alimente l'état interne des cellules LSTM via les portes d'entrée

4 matrices de poids



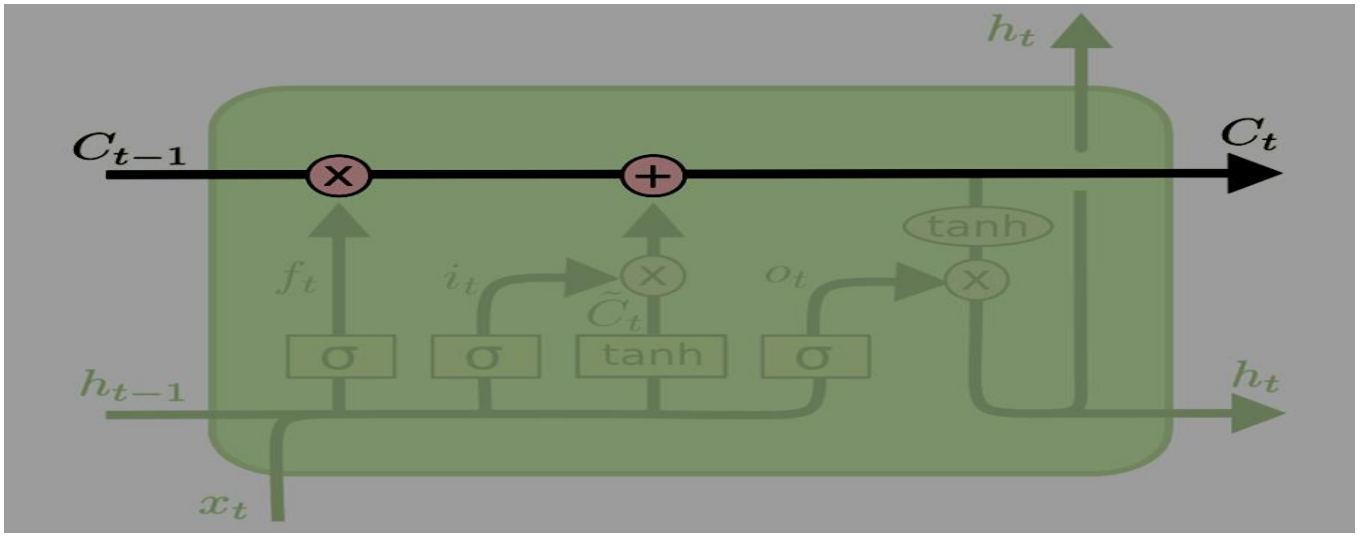
Wf : pondère l'entrée de la porte d'oubli (forget gate)
Wi : pondère l'entrée de la porte d'entrée (input gate)
WC : pondère les données qui vont se combiner à la porte d'entrée pour mettre à jour l'état de la cellule (cell state)
Wo : pondère l'entrée de la porte de sortie (output gate)

Structure d'un bloc LSTM



LSTM

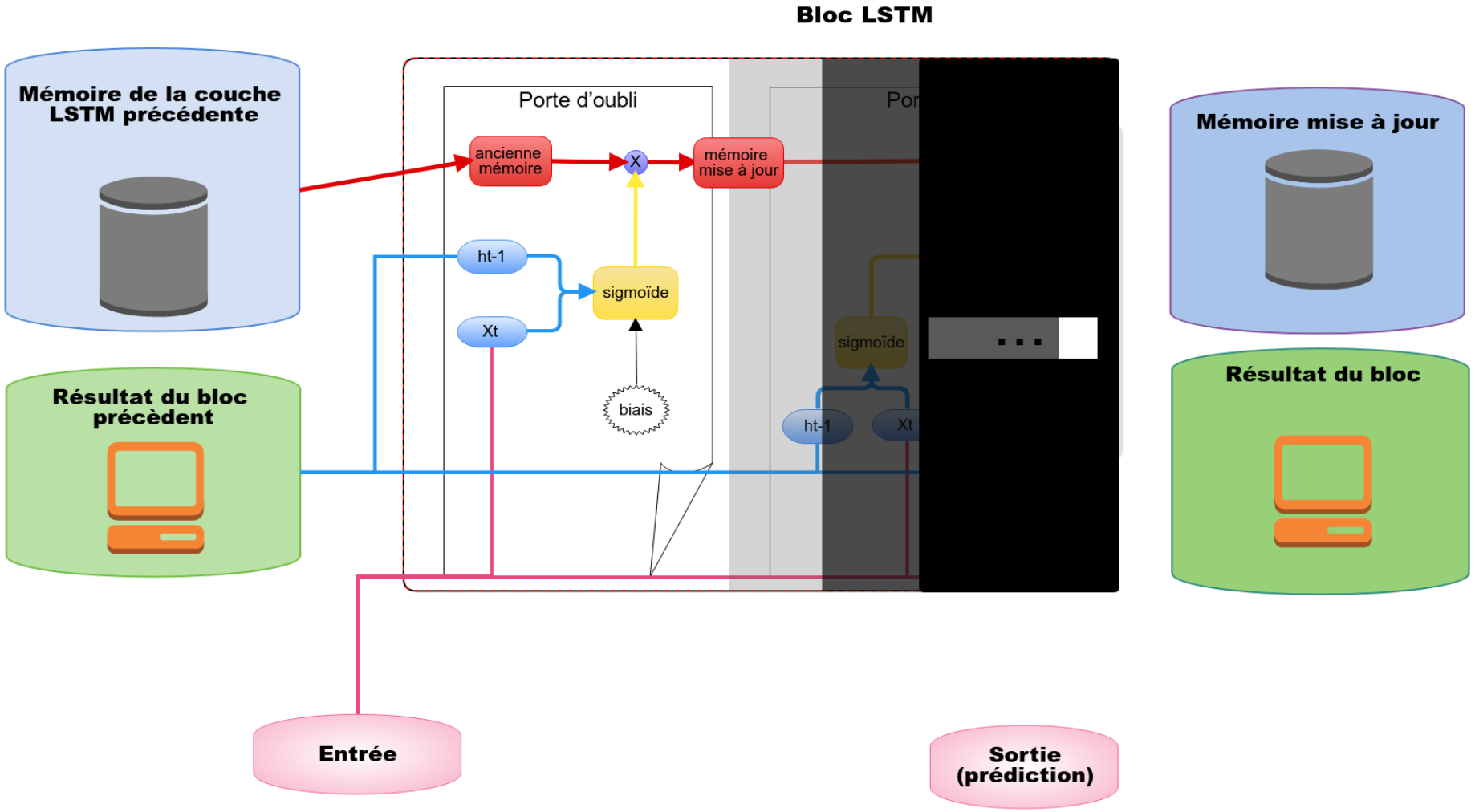
- La clé des LSTM est l'état de la cellule, la ligne horizontale qui traverse le haut du diagramme.
- L'état de la cellule est un peu comme un tapis roulant. Il court tout au long de la chaîne, avec seulement quelques interactions linéaires mineures. Il est très facile pour l'information de circuler sans changement.



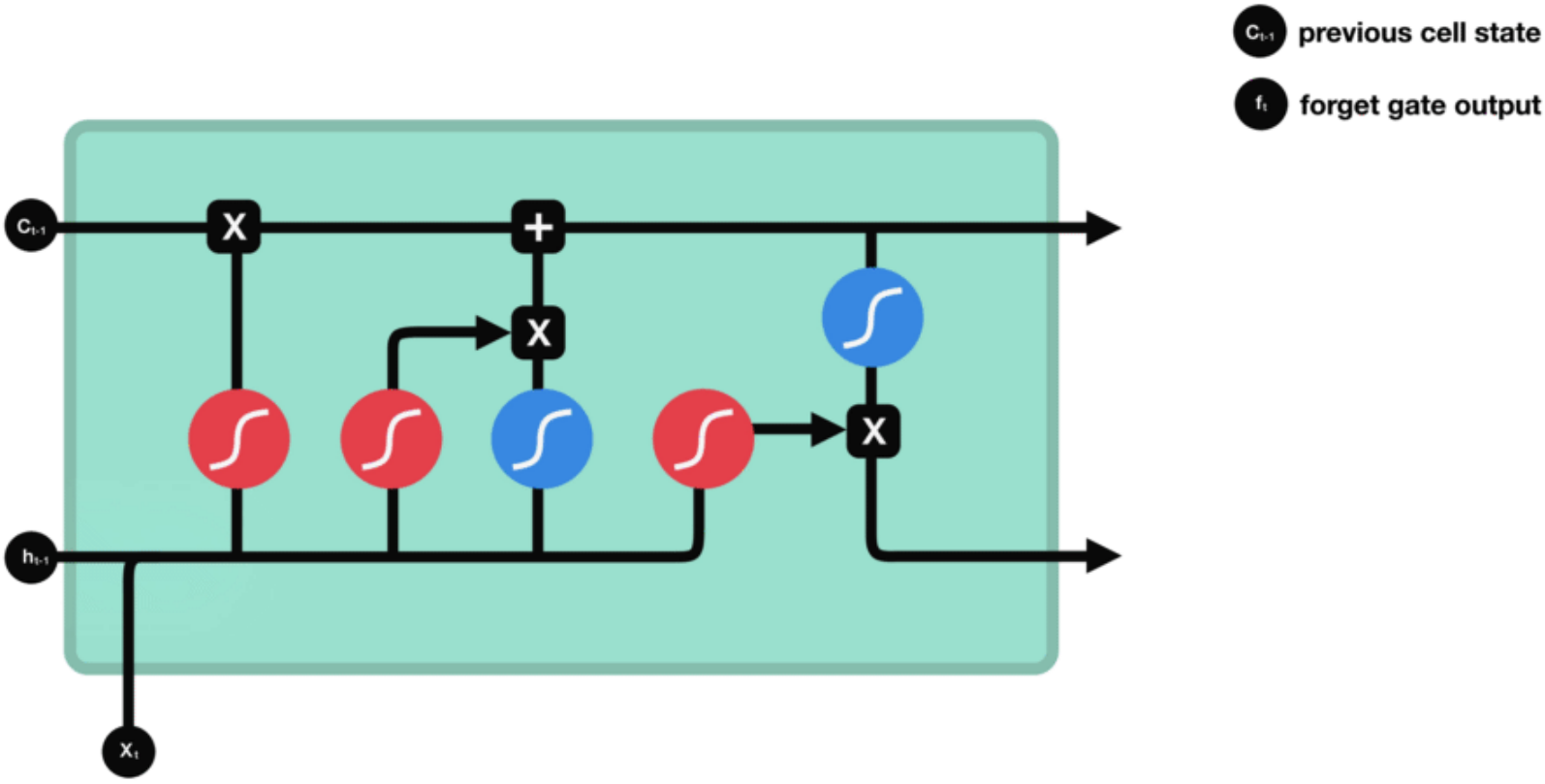
Porte d'oubli

- La première étape de notre LSTM consiste à décider quelles informations nous allons jeter de l'état de la cellule.
- Cette décision est prise par une couche sigmoïde appelée «porte d'oubli».

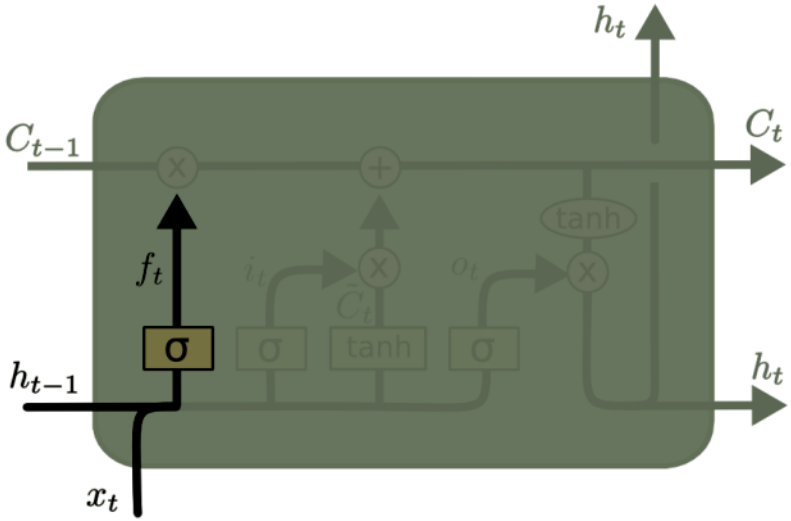
Porte d'oubli



Porte d'oubli



Forget



sigmoïde

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

La porte d'oubli va se charger de filtrer les informations contenues dans la cellule mémoire précédente. Parmis ces informations, certaines vont être plus pertinentes que d'autre. Pour décider quelles valeurs vont être autorisées à passer, différents calculs mathématiques sont utilisés

x_t correspond aux données entrantes dans le réseau de neurones, ***ht-1*** correspond à la valeur de sortie qui a été prédite par la couche LSTM précedente.

[ht-1,xt] désigne la **concaténation** des tableaux *xt* et *ht-1*. Comme dans un réseau de neurones classique, Wf correspond au poids des neurones et **bf** correspond au biais

Porte oubli

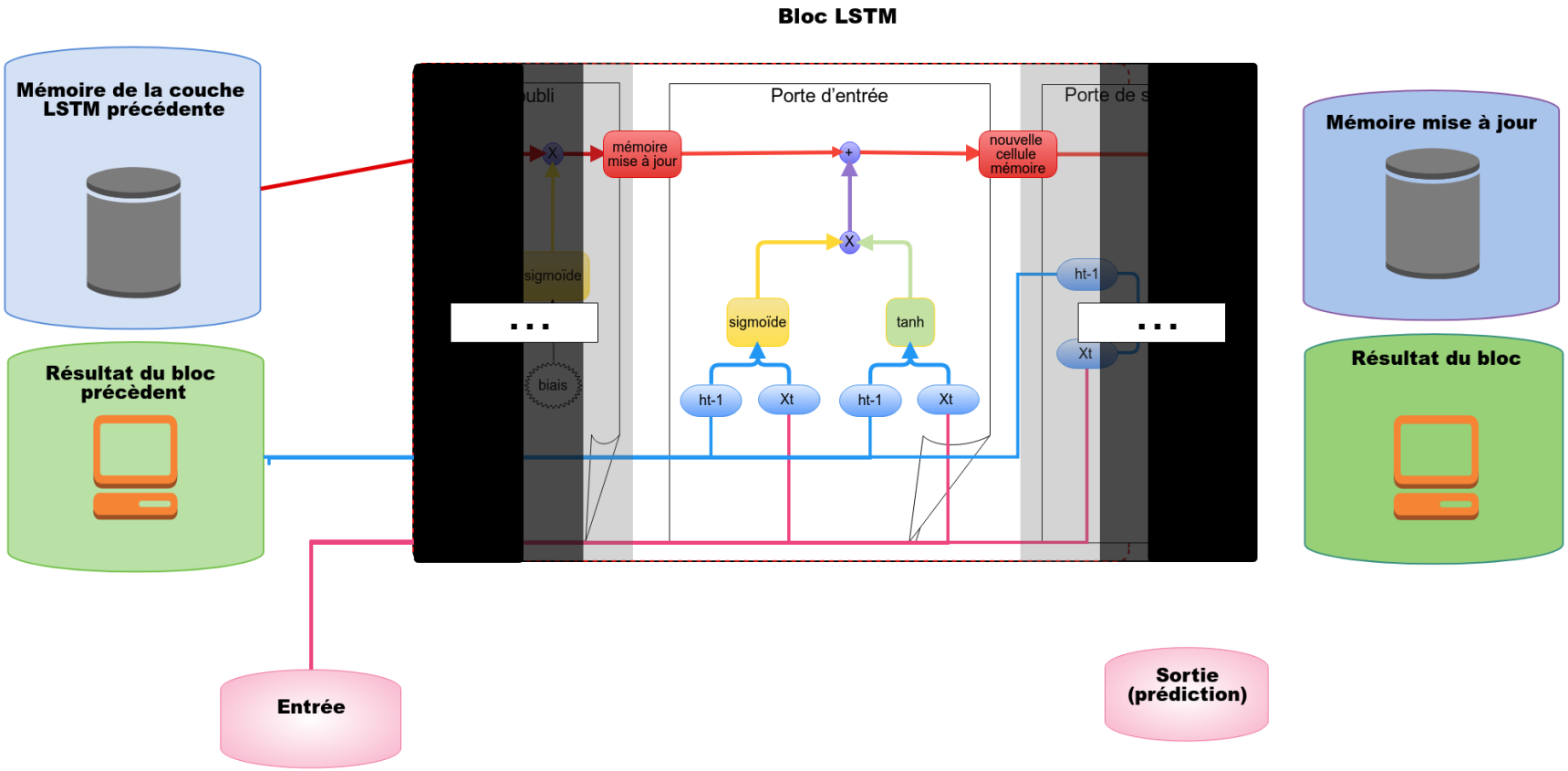
- f_t est donc un tableau de valeurs comprises entre 0 et 1. Pour savoir quelles valeurs de la cellule mémoire précédente C_{t-1} vont être autorisées à passer, nous allons appliquer la multiplication terme à terme entre f_t et C_{t-1}.

$$C_1 = f_t \times C_{t-1}$$

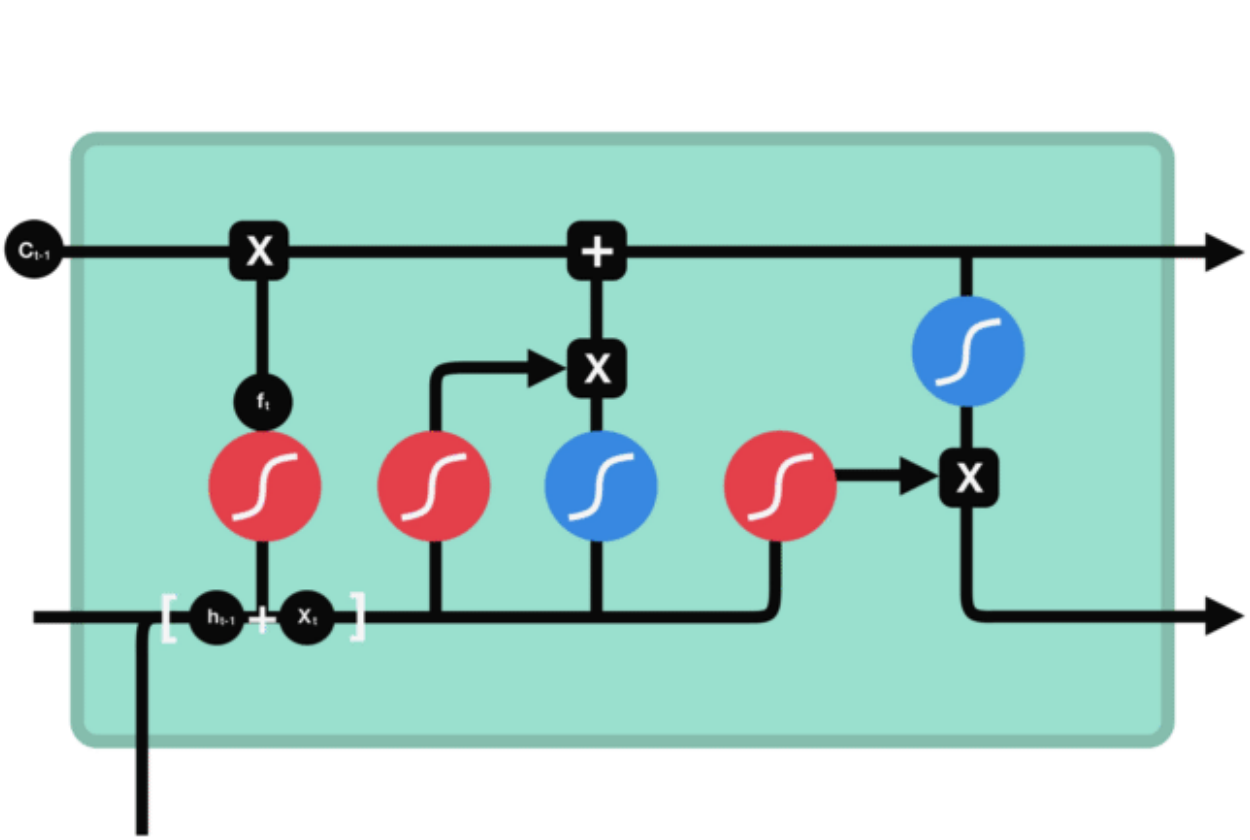
mise à jour de la cellule mémoire

- N.B: La porte d'oubli se charge uniquement de filtrer les anciennes valeurs contenues dans la cellule mémoire précédente.

Porte d'entrée



Porte d'entrée



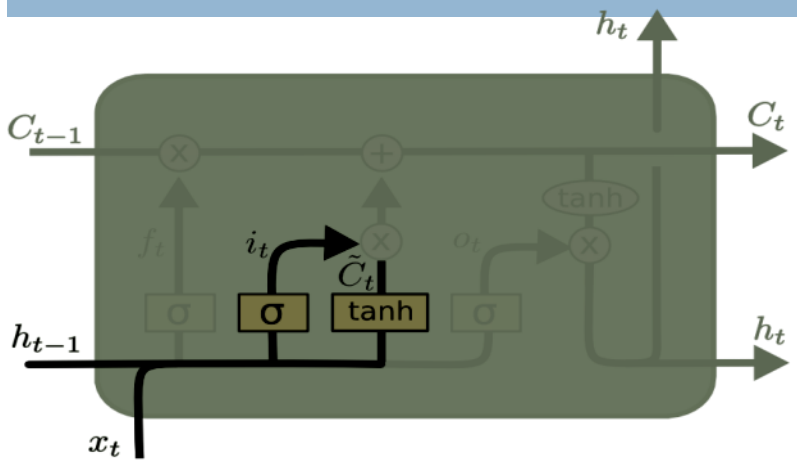
C_{t-1} previous cell state

f_t forget gate output

i_t input gate output

$\tilde{C}_t$ candidate

Porte d'entrée



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

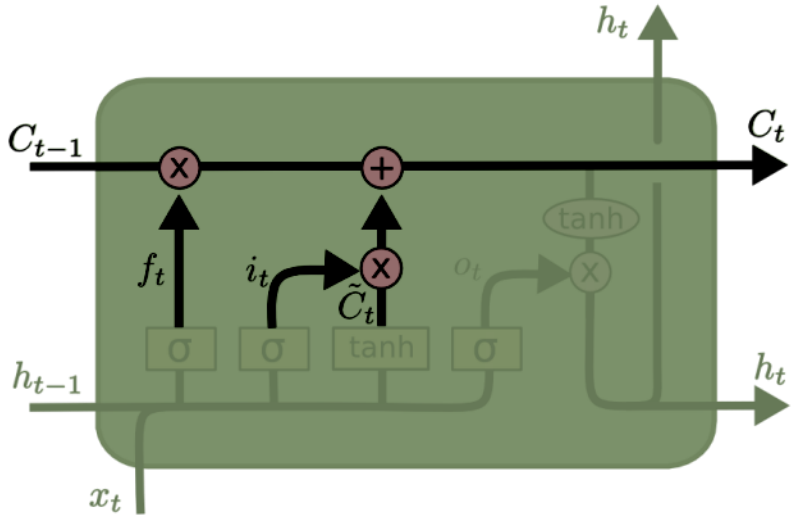
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

La porte d'entrée va se charger de décider quelles **nouvelles valeurs** provenant de *xt* (valeurs entrantes du LSTM) vont être autorisées à passer.

Certaines entrées vont être plus pertinentes que d'autres. Celles qui sont pertinentes vont être stockées dans la cellule mémoire qui va servir à la prise de décision à l'instant t et peut-être pour l'instant *t+1* si la porte d'oubli lui autorise l'accès.

Dans un premier temps, une fonction sigmoïde est nécessaire pour décider quelles nouvelles entrées seront susceptibles d'être stockées dans la cellule mémoire. *it* sera un tableau et chacune de ses valeurs sera comprise entre 0 et 1. Dans un second temps, nous allons faire la même chose sauf que la fonction d'activation utilisée est la fonction tangente hyperbolique (c'est une sigmoïde centré entre -1 et 1).

Update mémoire

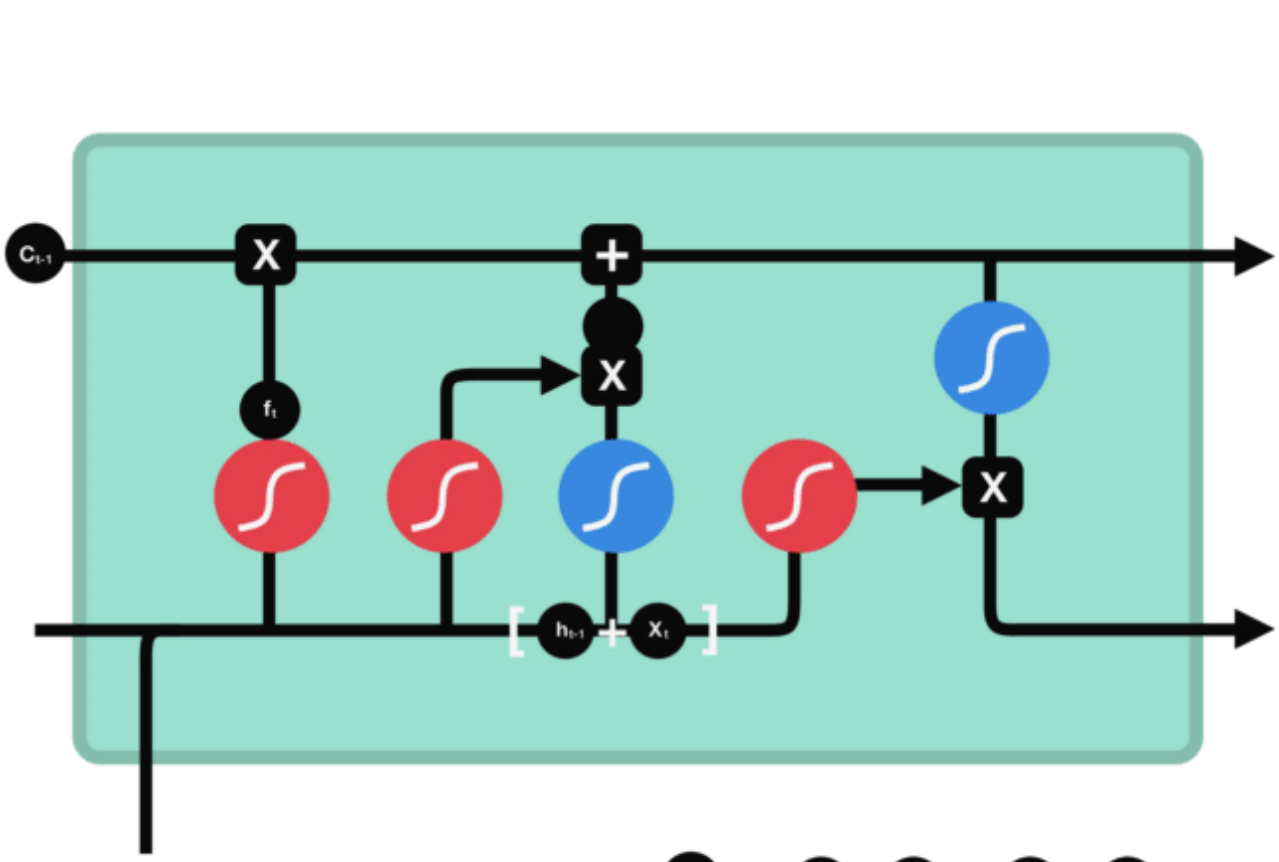


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

C~$_t$ est un tableau dont chaque valeur sera comprise entre -1 et 1. C~$_t$ contient les valeurs candidates à être stockées dans la cellule mémoire. La multiplication terme à terme entre i_t et C~$_t$ nous permettra de savoir quelles valeurs vont devoir être stockées dans la cellule mémoire.

Si le résultat est proche de 1 alors nous allons stocker la valeur correspondante dans la cellule mémoire, au contraire si le résultat est proche de 0, l'information sera bloquée. La nouvelle mémoire sera donc …. voir formule en haut

Mise à jour de l'état de la cellule d'un LSTM



C_{t-1} previous cell state

f_t forget gate output

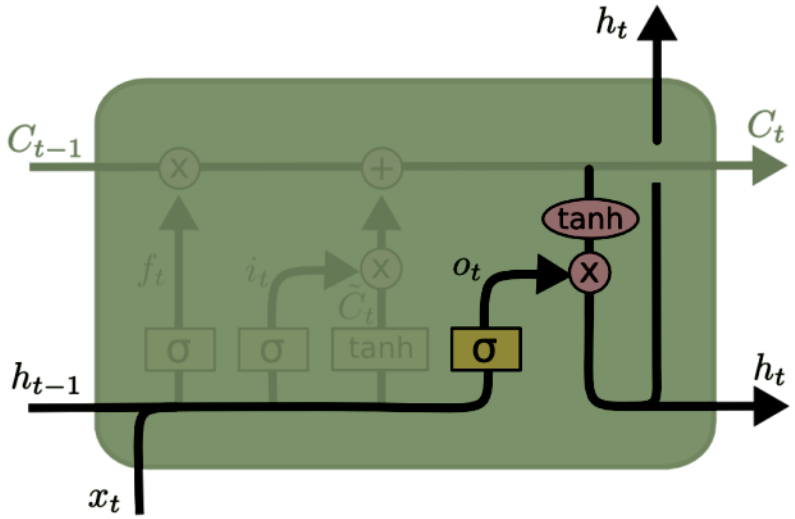
i_t input gate output

$\tilde{C}_t$ candidate

C_t new cell state

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Porte de sortie



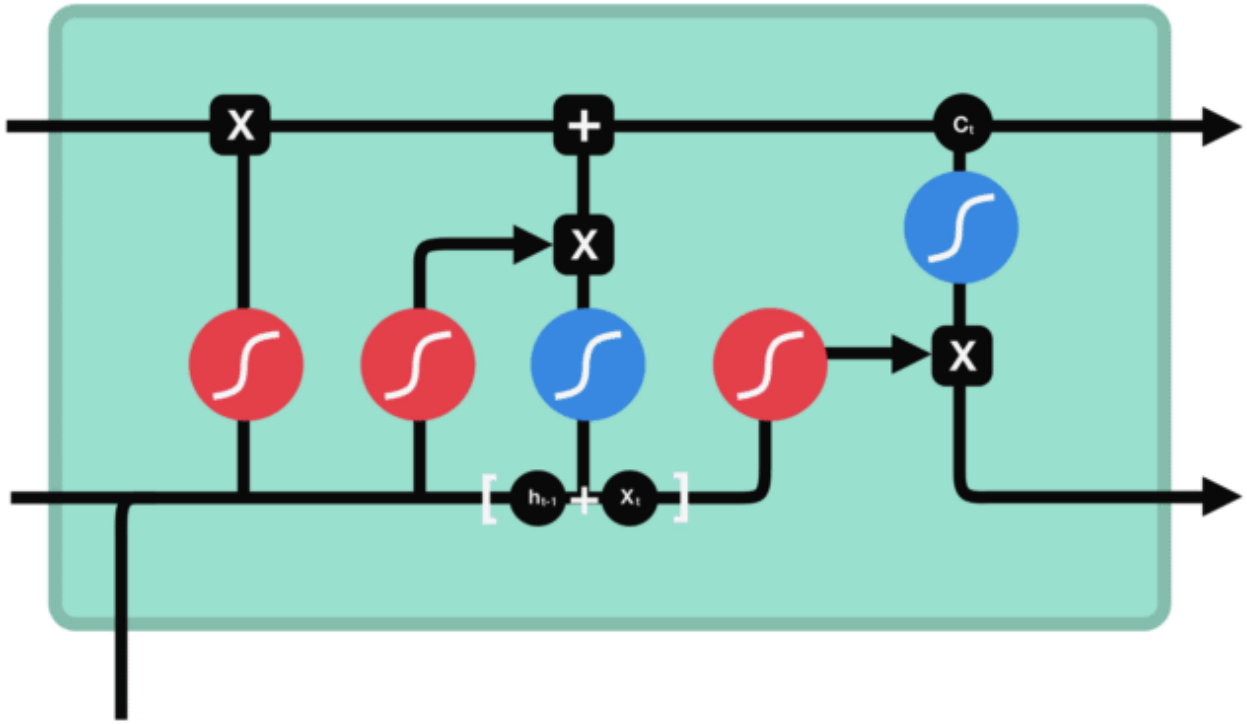
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

en fonction de l'entrée (*xt*), fournir une décision *ht* à la fois pour le bloc LSTM suivant ainsi que pour l'utilisateur. La prise de décision *ht* est calculée de la sorte :

La sortie *ht* va dépendre des informations présentes dans la cellule mémoire C*t* ainsi que des informations entrantes *xt* dans la couche LSTM.

Porte de sortie



c_{t-1} previous cell state

f_t forget gate output

i_t input gate output

$\tilde{c}_t$ candidate

c_t new cell state

o_t output gate output

h_t hidden state

Forward Pass

1. Basically very similar to the vanilla RNN forward pass
   1. But it's a lot more complicated
   2. Can you do the backward pass by yourself?
      1. Quick quiz, get a white sheet of paper. Write your student name.
      2. We are going to give you 10 minutes
      3. No discussion
      4. 10% of your final grades

Input Gates

$$a_\iota^t = \sum_{i=1}^{I} w_{i\iota} x_i^t + \sum_{h=1}^{H} w_{h\iota} b_h^{t-1} + \sum_{c=1}^{C} w_{c\iota} s_c^{t-1}$$
$$b_\iota^t = f(a_\iota^t)$$

Forget Gates

$$a_\phi^t = \sum_{i=1}^{I} w_{i\phi} x_i^t + \sum_{h=1}^{H} w_{h\phi} b_h^{t-1} + \sum_{c=1}^{C} w_{c\phi} s_c^{t-1}$$
$$b_\phi^t = f(a_\phi^t)$$

Cells

$$a_c^t = \sum_{i=1}^{I} w_{ic} x_i^t + \sum_{h=1}^{H} w_{hc} b_h^{t-1}$$
$$s_c^t = b_\phi^t s_c^{t-1} + b_\iota^t g(a_c^t)$$

Output Gates

$$a_\omega^t = \sum_{i=1}^{I} w_{i\omega} x_i^t + \sum_{h=1}^{H} w_{h\omega} b_h^{t-1} + \sum_{c=1}^{C} w_{c\omega} s_c^t$$
$$b_\omega^t = f(a_\omega^t)$$

Cell Outputs

$$b_c^t = b_\omega^t h(s_c^t)$$

Input Gates

$$a_\iota^t = \sum_{i=1}^{I} w_{i\iota} x_i^t + \sum_{h=1}^{H} w_{h\iota} b_h^{t-1} + \sum_{c=1}^{C} w_{c\iota} s_c^{t-1}$$

$$b_\iota^t = f(a_\iota^t)$$

Forget Gates

$$a_\phi^t = \sum_{i=1}^{I} w_{i\phi} x_i^t + \sum_{h=1}^{H} w_{h\phi} b_h^{t-1} + \sum_{c=1}^{C} w_{c\phi} s_c^{t-1}$$

$$b_\phi^t = f(a_\phi^t)$$

Cells

$$a_c^t = \sum_{i=1}^{I} w_{ic} x_i^t + \sum_{h=1}^{H} w_{hc} b_h^{t-1}$$

$$s_c^t = b_\phi^t s_c^{t-1} + b_\iota^t g(a_c^t)$$

Output Gates

$$a_\omega^t = \sum_{i=1}^{I} w_{i\omega} x_i^t + \sum_{h=1}^{H} w_{h\omega} b_h^{t-1} + \sum_{c=1}^{C} w_{c\omega} s_c^t$$

$$b_\omega^t = f(a_\omega^t)$$

Cell Outputs

$$b_c^t = b_\omega^t h(s_c^t)$$

Backward Pass

1. Just kidding! The math to get the backward pass should be very similar to the one used in vanilla RNN backward pass
   1. But it's a lot more complicated, too...
   2. We are not going to derive that in the class

$$\epsilon_c^t \stackrel{\text{def}}{=} \frac{\partial O}{\partial b_c^t} \qquad \epsilon_s^t \stackrel{\text{def}}{=} \frac{\partial O}{\partial s_c^t}$$

Cell Outputs

$$\epsilon_c^t = \sum_{k=1}^{K} w_{ck}\delta_k^t + \sum_{h=1}^{H} w_{ch}\delta_h^{t+1}$$

Output Gates

$$\delta_\omega^t = f'(a_\omega^t)\sum_{c=1}^{C} h(s_c^t)\epsilon_c^t$$

States

$$\epsilon_s^t = b_\omega^t h'(s_c^t)\epsilon_c^t + b_\phi^{t+1}\epsilon_s^{t+1} + w_{c\iota}\delta_\iota^{t+1} + w_{c\phi}\delta_\phi^{t+1} + w_{c\omega}\delta_\omega^t$$

Cells

$$\delta_c^t = b_\iota^t g'(a_c^t)\epsilon_s^t$$

Forget Gates

$$\delta_\phi^t = f'(a_\phi^t)\sum_{c=1}^{C} s_c^{t-1}\epsilon_s^t$$

Input Gates

$$\delta_\iota^t = f'(a_\iota^t)\sum_{c=1}^{C} g(a_c^t)\epsilon_s^t$$

$$\epsilon_c^t \stackrel{\text{def}}{=} \frac{\partial O}{\partial b_c^t} \qquad \epsilon_s^t \stackrel{\text{def}}{=} \frac{\partial O}{\partial s_c^t}$$

Cell Outputs

$$\epsilon_c^t = \sum_{k=1}^{K} w_{ck}\delta_k^t + \sum_{h=1}^{H} w_{ch}\delta_h^{t+1}$$

Output Gates

$$\delta_\omega^t = f'(a_\omega^t)\sum_{c=1}^{C} h(s_c^t)\epsilon_c^t$$

States

$$\epsilon_s^t = b_\omega^t h'(s_c^t)\epsilon_c^t + b_\phi^{t+1}\epsilon_s^{t+1} + w_{c\iota}\delta_\iota^{t+1} + w_{c\phi}\delta_\phi^{t+1} + w_{c\omega}\delta_\omega^t$$

Cells

$$\delta_c^t = b_\iota^t g'(a_c^t)\epsilon_s^t$$

Forget Gates

$$\delta_\phi^t = f'(a_\phi^t)\sum_{c=1}^{C} s_c^{t-1}\epsilon_s^t$$

Input Gates

$$\delta_\iota^t = f'(a_\iota^t)\sum_{c=1}^{C} g(a_c^t)\epsilon_s^t$$

```
1  import numpy as np
2  import tensorflow as tf
3  from tensorflow.keras.models import Sequential
4  from tensorflow.keras.layers import Dense
5  from tensorflow.keras.layers import Dropout
6  from tensorflow.keras.layers import LSTM
7  from tensorflow.keras.callbacks import ModelCheckpoint
8  from tensorflow.keras.utils import to_categorical
9
   filename = "wonderland.txt"
   raw_text = open(filename, 'r', encoding='utf-8').read()
   raw_text = raw_text.lower()

   # create mapping of unique chars to integers
   chars = sorted(list(set(raw_text)))
   char_to_int = dict((c, i) for i, c in enumerate(chars))
```

```
...
n_chars = len(raw_text)
n_vocab = len(chars)
print "Total Characters: ", n_chars
print "Total Vocab: ", n_vocab

# prepare the dataset of input to output pairs encoded as integers
seq_length = 100
dataX = []
dataY = []
for i in range(0, n_chars - seq_length, 1):
seq_in = raw_text[i:i + seq_length]
seq_out = raw_text[i + seq_length]
dataX.append([char_to_int[char] for char in seq_in])
dataY.append(char_to_int[seq_out])
n_patterns = len(dataX)
print "Total Patterns: ", n_patterns
```

```
...
# reshape X to be [samples, time steps, features]
X = np.reshape(dataX, (n_patterns, seq_length, 1))
# normalize
X = X / float(n_vocab)
# one hot encode the output variable
y = to_categorical(dataY)

# define the LSTM model
model = Sequential()
model.add(LSTM(256, input_shape=(X.shape[1], X.shape[2])))
model.add(Dropout(0.2))
model.add(Dense(y.shape[1], activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam')
```

```
...
# define the checkpoint
filepath="weights-improvement-{epoch:02d}-{loss:.4f}.hdf5"
checkpoint = ModelCheckpoint(filepath, monitor='loss', verbose=1,
save_best_only=True, mode='min')
callbacks_list = [checkpoint]

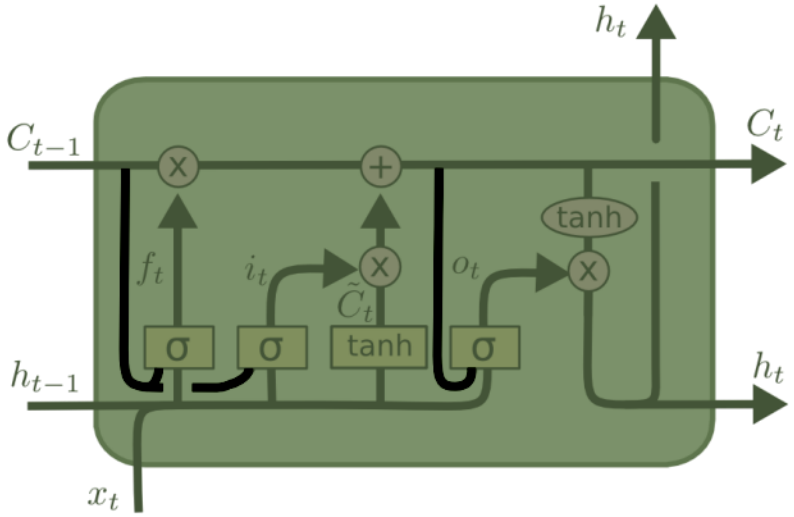

model.fit(X, y, epochs=20, batch_size=128, callbacks=callbacks_list)
```

Génération de texte

Variants on Long Short Term Memory

Variants on Long Short Term Memory

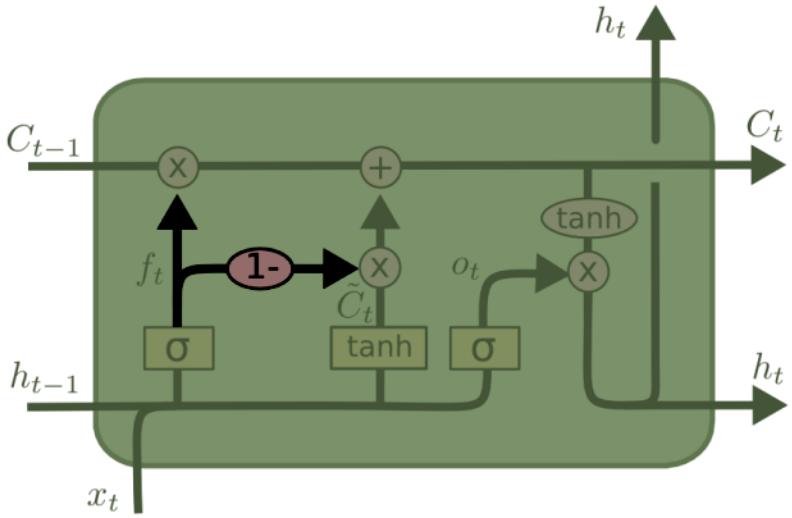
- One popular LSTM variant, introduced by Gers & Schmidhuber (2000), is adding "peephole connections." This means that we let the gate layers look at the cell state.



$$f_t = \sigma\left(W_f \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_f\right)$$
$$i_t = \sigma\left(W_i \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_i\right)$$
$$o_t = \sigma\left(W_o \cdot [\boldsymbol{C_t}, h_{t-1}, x_t] \; + \; b_o\right)$$

Variants on Long Short Term Memory

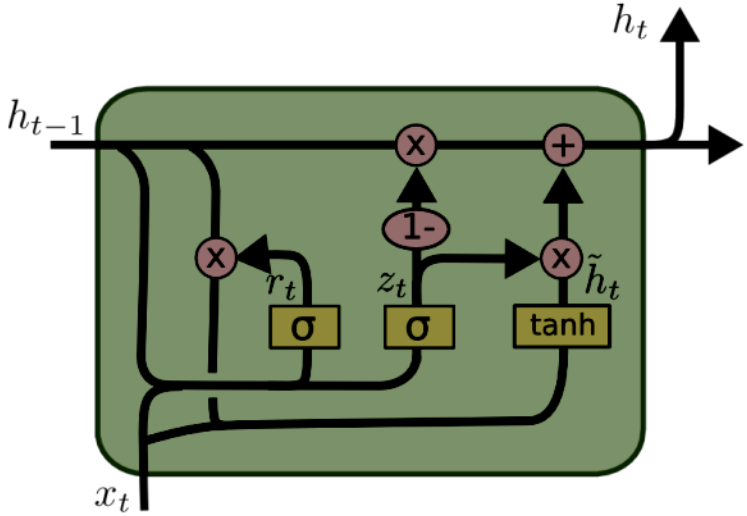
- Another variation is to use coupled forget and input gates. Instead of separately deciding what to forget and what we should add new information to, we make those decisions together.
- We only forget when we're going to input something in its place. We only input new values to the state when we forget something older.



$$C_t = f_t * C_{t-1} + (1 - \boldsymbol{f_t}) * \tilde{C}_t$$

Variants on Long Short Term Memory

- A slightly more dramatic variation on the LSTM is the Gated Recurrent Unit, or GRU, introduced by Cho, et al. (2014). It combines the forget and input gates into a single "update gate." It also merges the cell state and hidden state, and makes some other changes. The resulting model is simpler than standard LSTM models, and has been growing increasingly popular.



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

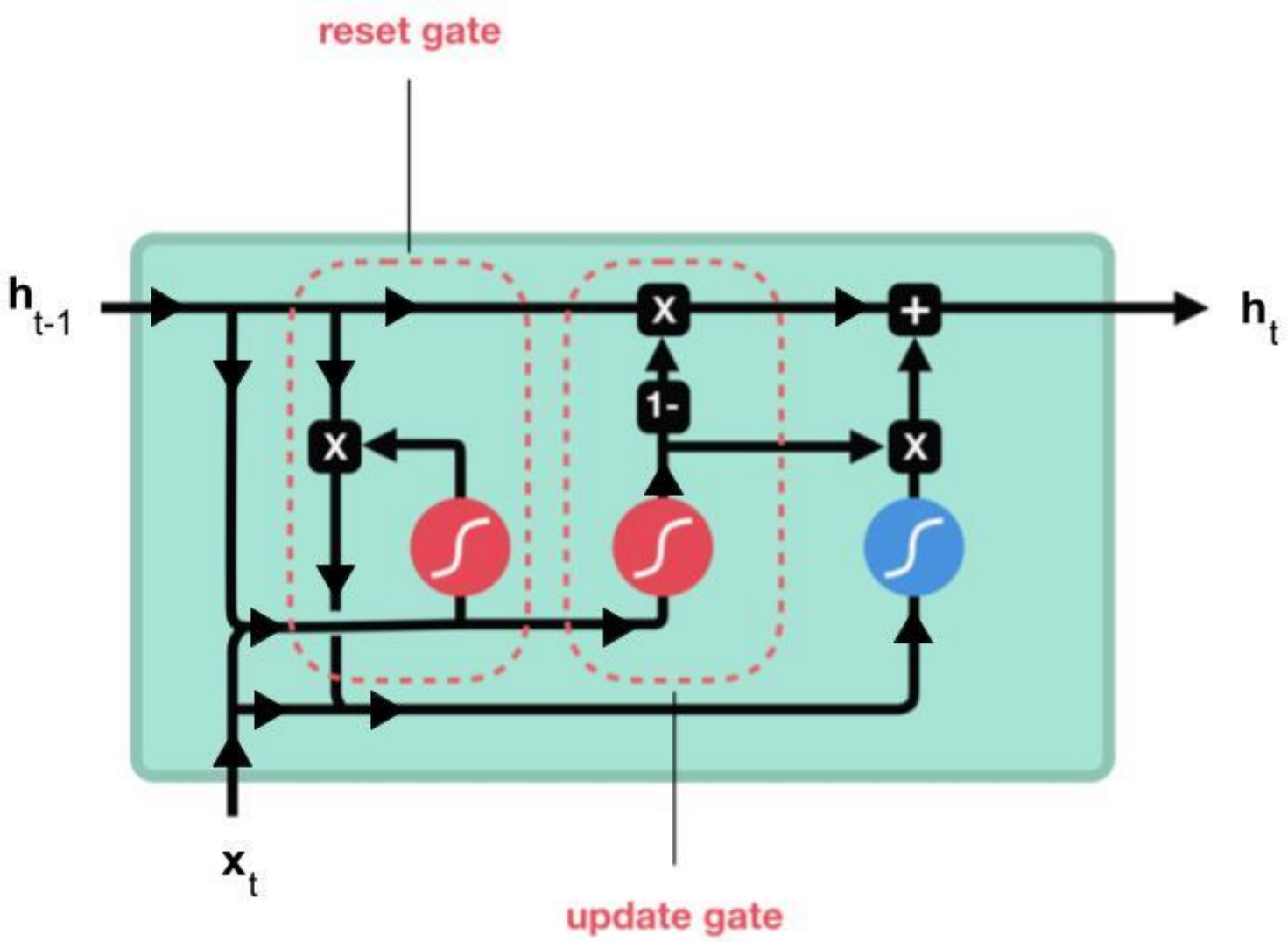
Comparaison & test

- https://arxiv.org/pdf/1503.04069.pdf
- http://proceedings.mlr.press/v37/jozefowicz15.pdf

GRU

- **GRU**, *Gated Recurrent Unit*, pour sa part dispose de deux portes et un état en sortie :
  - Reset gate (porte de reset)
  - Update gate (porte de mise à jour)
  - Cell state (état de la cellule)

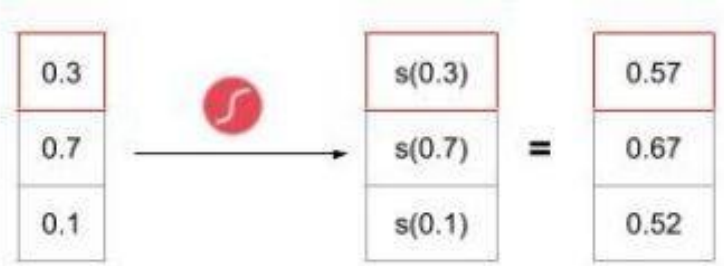
GRU



Opérations dans un GRU

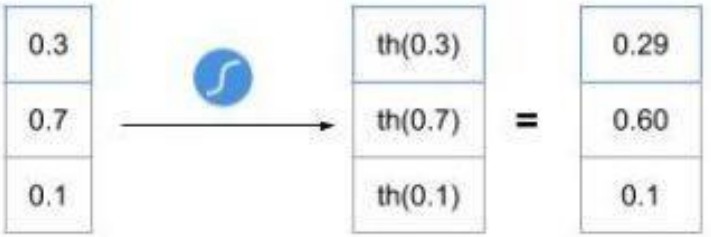


Sigmoïde (notée s)
Fonction à appliquer



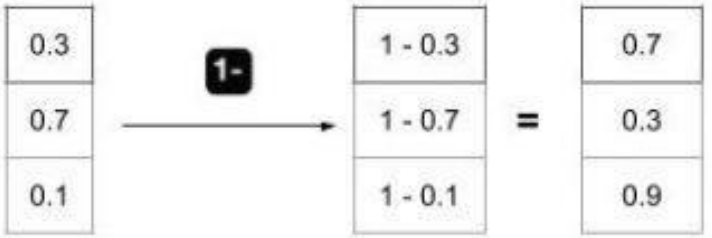


Tanh (notée th)
Fonction à appliquer





Symétrique
Chaque coordonnée est remplacée par 1 - la coordonnée



La seule différence avec le LSTM est « 1 -«

Opérations dans un GRU



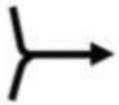
Multiplication
Multiplier les coordonnées face à face

0.3		0.2	=	0.06
0.7	×	0.4		0.28
0.1		0.4		0.04



Addition
Additionner les coordonnées face à face

0.3		0.2	=	0.5
0.7	+	0.4		1.1
0.1		0.4		0.5



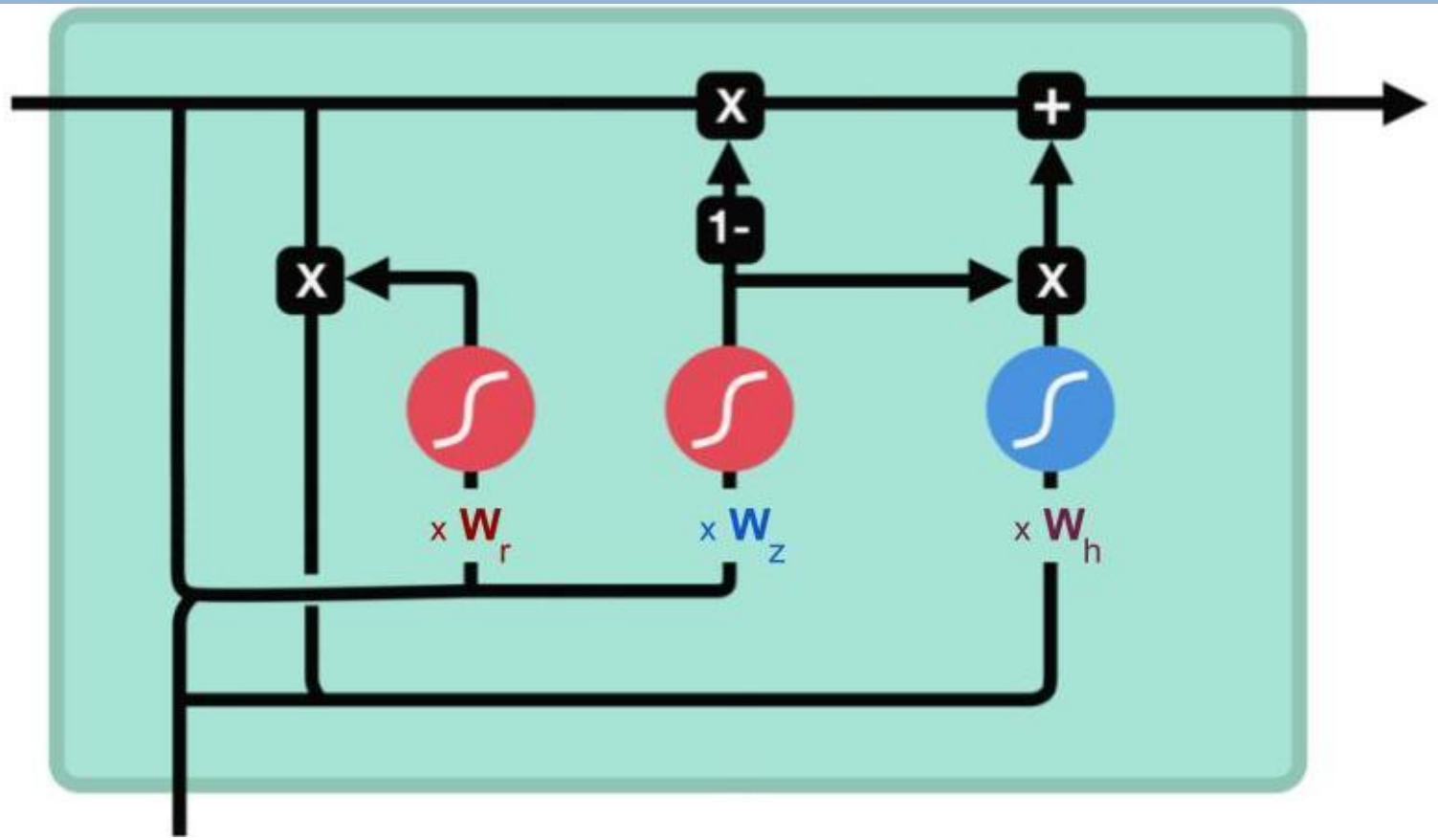
Concaténation
Mettre les deux vecteurs bout-à-bout

0.3		0.3
0.7		0.7
0.1		0.1
	→	0.2
0.2		0.4
0.4		0.4
0.4		

Fonctionnement

- l'apparition d'une nouvelle opération qui consiste à inverser un vecteur par rapport à 1 (on calcule 1 – le vecteur).
- Ceci a pour objectif d'inverser les conclusions d'une porte (par exemple, que tout ce qui a été jugé important devienne anodin et vice versa) : en logique, c'est une porte NOT.

Le GRU dispose de « 3 matrices de poids »



Wr : pondère l'entrée de la porte de reset (reset gate)

Wz : pondère l'entrée de la porte de mise à jour (update gate)

Wh : pondère les données qui vont se combiner pour définir l'état caché courant

to be continued